

MI AO API

Version 3.54

© 2022 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
3.50	Initial release	12/04/2020
3.51	Audio new architecture	07/19/2021
	Added procfs introduction	08/25/2021
3.52	Added MI_AO_GetAttr	09/08/2021
	Modified version: Audio 3.0 Version starts from 3.50(3.0 to 3.49 are Audio 2.0 versions)	12/29/2021
3.53	Mochi series chip API version modified, Change keyword Dircet Path to Passthrough	02/17/2022
3.54	Added Maruko series chip info	03/29/2022

TABLE OF CONTENTS

REVISION HISTORY.....	i
TABLE OF CONTENTS.....	ii
1. 概述.....	1
1.1. 模块说明.....	1
1.2. Audio Codec 框图.....	2
1.2.1 Muffin 系列	2
1.2.2 Mochi 系列	3
1.2.3 Maruko 系列	4
1.3. Audio Codec 器件说明.....	4
1.4. Audio Codec 差异说明.....	5
1.4.1 Muffin 系列	5
1.4.2 Mochi 系列	6
1.4.3 Maruko 系列	6
1.5. API 关键字说明.....	6
2. API 参考.....	15
2.1. MI_AO_Open.....	16
2.2. MI_AO_OpenWithCfgFile.....	18
2.3. MI_AO_Close.....	19
2.4. MI_AO_AttachIf.....	20
2.5. MI_AO_DetachIf.....	21
2.6. MI_AO_Write.....	22
2.7. MI_AO_Start.....	23
2.8. MI_AO_Stop.....	23
2.9. MI_AO_Pause.....	24
2.10. MI_AO_Resume.....	25
2.11. MI_AO_SetVolume.....	25
2.12. MI_AO_GetVolume.....	26
2.13. MI_AO_SetMute.....	27
2.14. MI_AO_GetMute.....	28
2.15. MI_AO_SetIfVolume.....	28
2.16. MI_AO_GetIfVolume.....	29
2.17. MI_AO_SetIfMute.....	30
2.18. MI_AO_GetIfMute.....	31
2.19. MI_AO_SetI2SConfig.....	31
2.20. MI_AO_GetI2SConfig.....	32
2.21. MI_AO_AdjustSpeed.....	33
2.22. MI_AO_GetTimestamp.....	34
2.23. MI_AO_GetLatency.....	34
2.24. MI_AO_InitDev.....	35
2.25. MI_AO_DeInitDev.....	36
2.26. MI_AO_GetAttr.....	36
2.27. MI_AO_SetChannelMode.....	37
3. AO 数据类型.....	39

3.1.	MI_AUDIO_DEV.....	39
3.2.	MI_AUDIO_Format_e.....	40
3.3.	MI_AUDIO_SoundMode_e.....	40
3.4.	MI_AUDIO_SampleRate_e.....	41
3.5.	MI_AO_ChannelMode_e.....	42
3.6.	MI_AO_Attr_t.....	43
3.7.	MI_AO_If_e.....	44
3.8.	MI_AO_GainFading_e.....	45
3.9.	MI_AUDIO_I2sMode_e.....	45
3.10.	MI_AUDIO_I2sBitWidth_e.....	46
3.11.	MI_AUDIO_I2sFormat_e.....	47
3.12.	MI_AUDIO_I2sMclk_e.....	47
3.13.	MI_AUDIO_I2sConfig_t.....	48
3.14.	MI_AO_InitParam_t.....	49
4.	错误码.....	50
5.	PROCFS 介绍.....	51
5.1.	cat.....	51
5.2.	echo.....	53

1. 概述

1.1. 模块说明

音频输出（Audio Output, AO）主要实现配置及启用音频输出设备、写入音频数据、以及音量设置等功能。

1.2. Audio Codec 框图

1.2.1 Muffin 系列

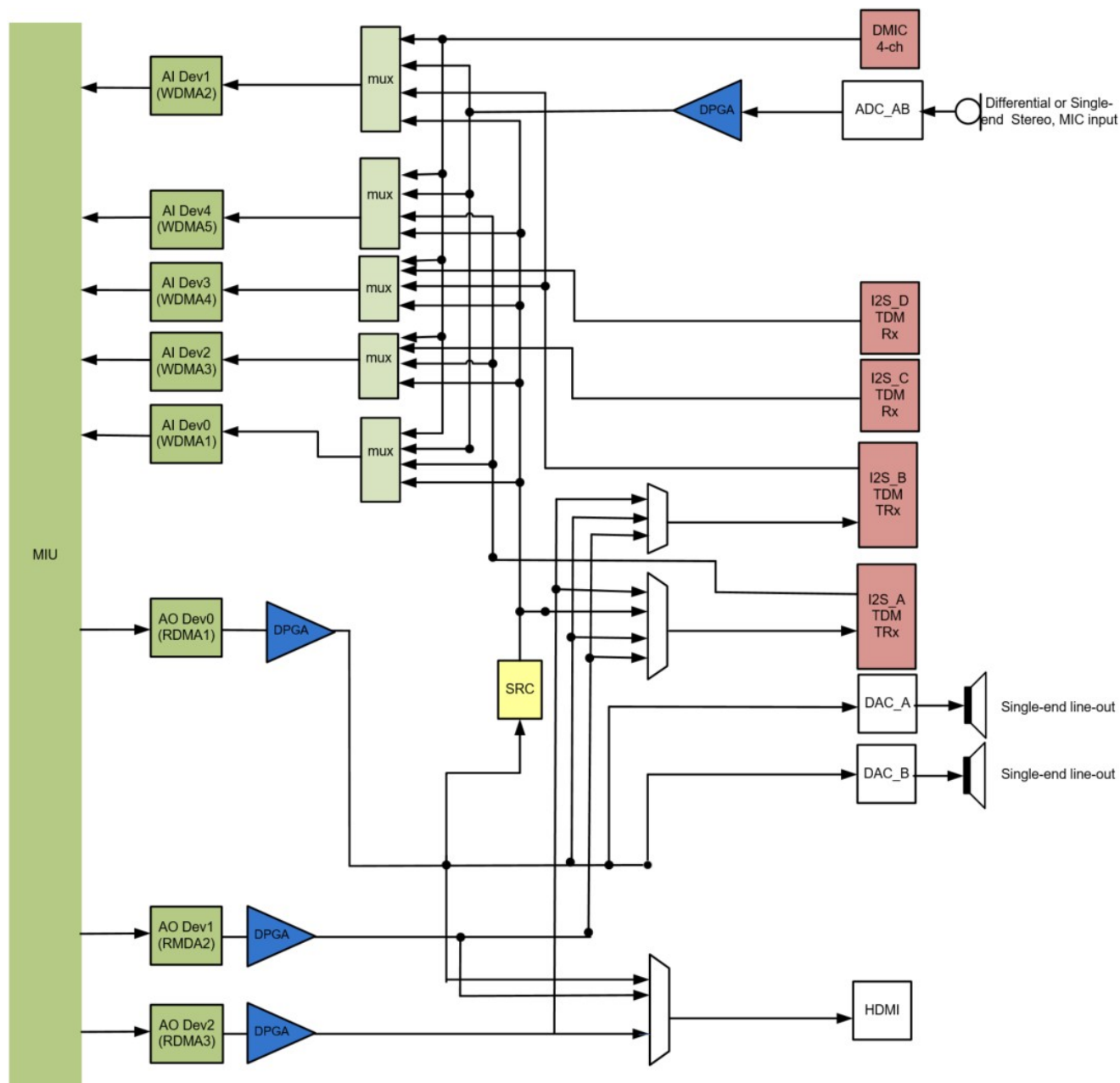


图 1-1: Muffin 系列芯片

Muffin 系列芯片的 audio codec 拥有如下资源：WDMA * 5, RDMA * 3, DMIC 接口(可支持 4Chn DMIC 信号) * 1, ADC(可支持 2Chn Amic/Linein) * 2, I2S-TDM RX * 4, I2S-TDM TX * 2, DAC(可支持 2Chn Lineout) * 2, SRC * 1, HDMI TX * 1。

1.2.2 Mochi 系列

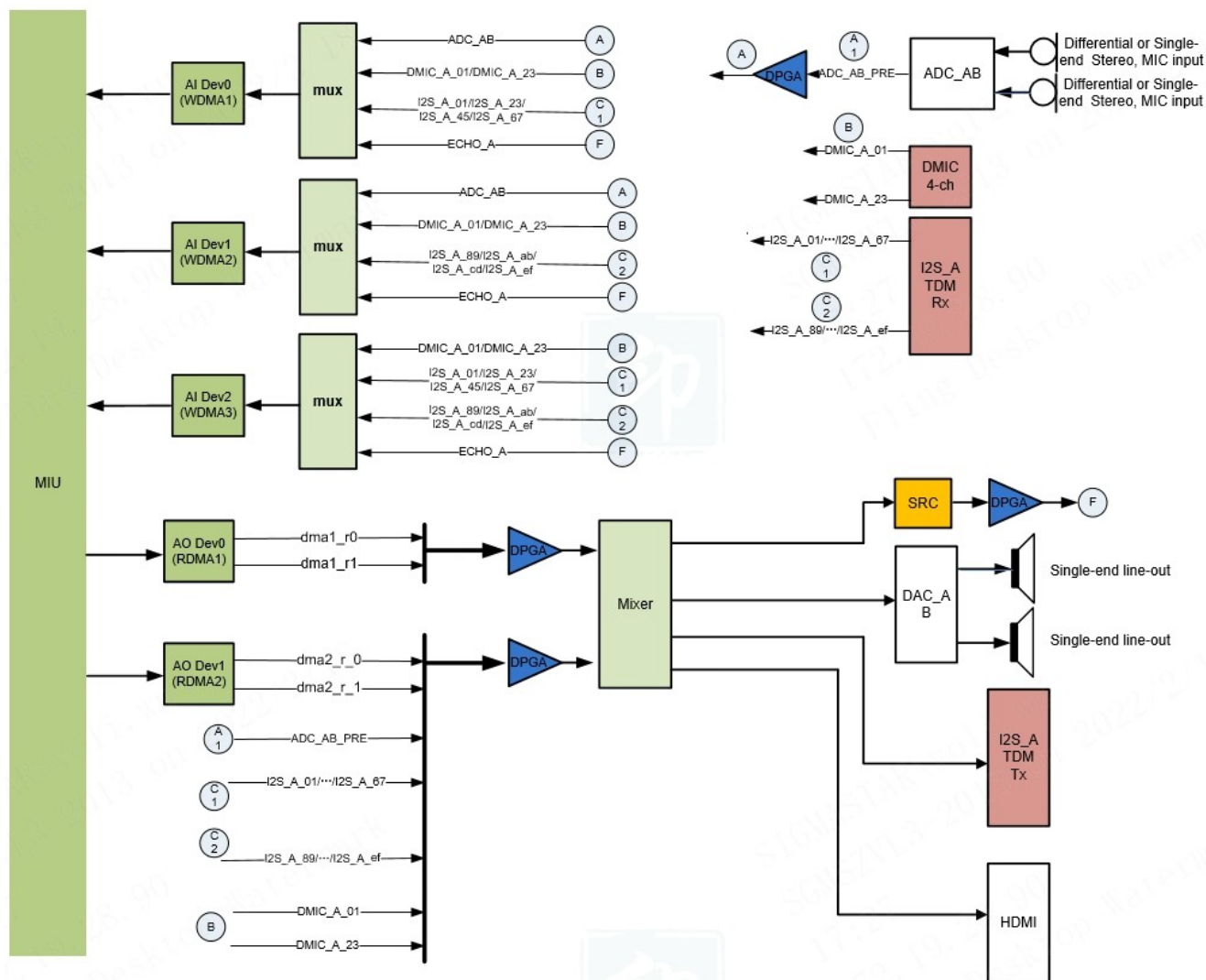


图 1-2: Mochi 系列芯片

Mochi 系列芯片的 audio codec 拥有如下资源：WDMA * 3, RDMA * 2, DMIC 接口(可支持 4Chn DMIC 信号) * 1, ADC(可支持 2Chn Amic/Linein) * 2, I2S-TDM RX(16 slot) * 1, DAC(可支持 2Chn Lineout) * 2, SRC * 1, HDMI TX * 1, I2S-TDM TX(16 slot, 但只有 2 路有效数据)。

1.2.3 Maruko 系列

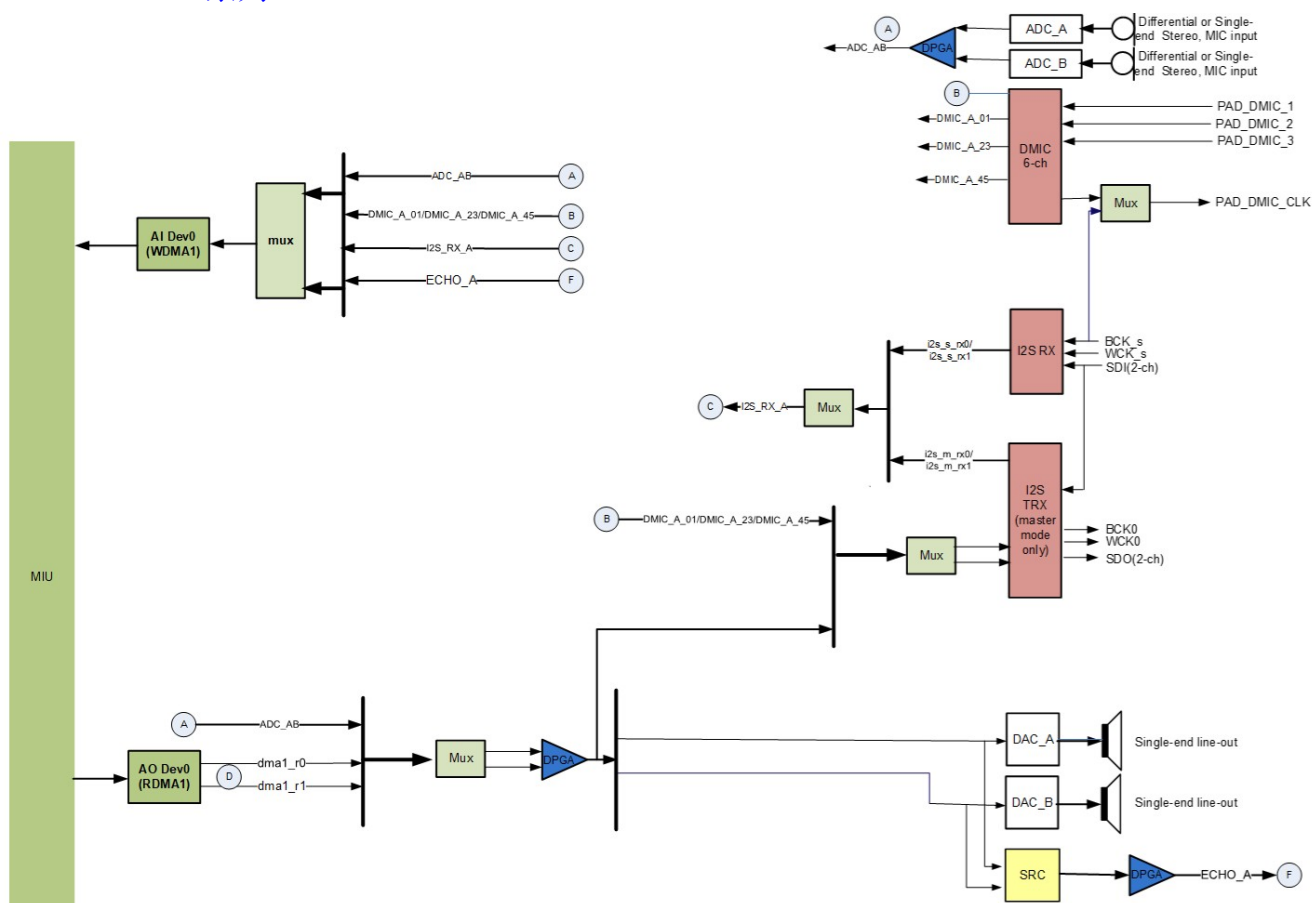


图 1-3: Maruko 系列芯片

Maruko 系列芯片的 audio codec 拥有如下资源：WDMA * 1, RDMA * 1, DMIC 接口(可支持 6Chn DMIC 信号) * 1, ADC(可支持 2Chn Amic/Linein) * 2, I2S RX(2 slot) * 2, DAC(可支持 2Chn Lineout) * 2, SRC * 1, I2S TX(2 slot) * 1。

1.3. Audio Codec 器件说明

- DMA

DMA 即 Direct Memory Access 直接存储器访问，DMA 传输将数据从一个地址空间复制到另一个地址空间，提供在外设和存储器之间或者存储器和存储器之间的高速数据传输。当 CPU 初始化这个传输动作，传输动作本身是由 DMA 控制器来实现和完成的。DMA 传输方式无需 CPU 直接控制传输，也没有中断处理方式那样保留现场和恢复现场过程，通过硬件为 RAM 和 IO 设备开辟一条直接传输数据的通道，使得 CPU 的效率大大提高。

- WDMA

WDMA 即 Direct Memory Access Writer 直接存储器访问写入器。

- RDMA

RDMA 即 Direct Memory Access Reader 直接存储器访问读取器。

- MUX

MUX 即 **multiplexer** 数据选择器，在多路数据传送过程中，能够根据需要将其中的任意多路选出来的电路。WDMA 前面的 Mux 为 WDMA 选择多个数据源，可支持选择 1/2/4 个数据源(数据源可以相同也可以不同)，每个数据源各有两声道，即选择 2/4/8 个声道的数据由 WDMA 写到 DRAM，起到多路开关的作用。而接近输出外设接口(如 I2S TX/HDMI/DAC 等)的 Mux 实现的是多选一的作用。

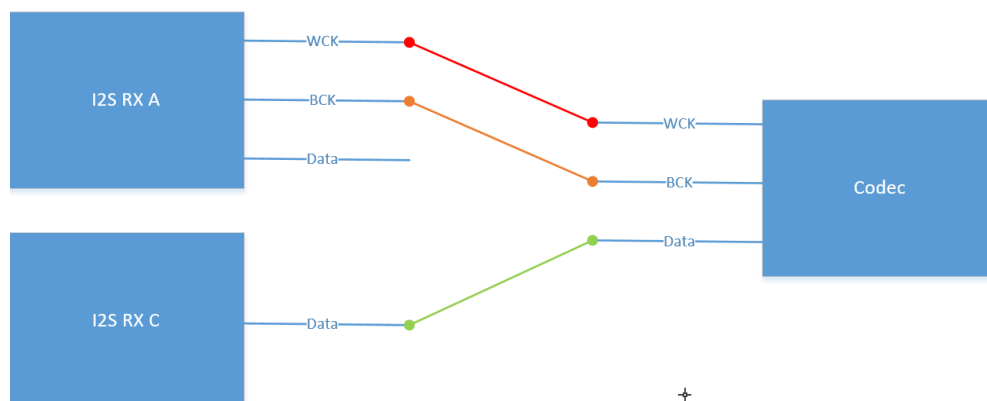
- **DPGA**
DPGA 即 **Digital Programmable Gain Amplifier** 数字可编程增益放大器，是一种通用性很强的放大器，其放大倍数可以根据需要用程序进行控制。
- **DMIC**
DMIC 即 **Digital Microphone Interface** 数字麦克风接口，audio codec 仅提供 DMIC 接口，并非完整的 DMIC。DMIC 接口提供 DMIC 工作所需要的时钟信号，接收从 DMIC 来的 PDM 信号
- **ADC**
ADC 即 **Analog Digital Conversion** 模拟数字转换，将模拟信号转变为数字信号电子元件。
- **I2S**
I2S 即 **Inter-IC Sound** 集成电路内置音频总线，是飞利浦公司为数字音频设备之间的音频数据传输而制定的一种总线标准。Sigmastar 的 I2S 总线仅支持标准 I2S 数据格式以及左对齐的 I2S 数据格式。同时也支持 TDM 即 **Time-Division Multiplexing** 时分复用技术，将不同的信号相互交织在不同的时间段内，沿着同一个信道传输，可同时支持 4/8/16 通道数据传输。
- **DAC**
DAC 即 **Digital Analog Conversion** 数字模拟转换，将数字信号转变为模拟信号电子元件。
- **SRC**
SRC 即 **Sample Rate Convert** 采样率转换，对语音数字信号进行采样率转换。
- **HDMI**
HDMI(High Definition Multimedia Interface, 高清多媒体接口), AO 输出接口，是一种全数字化视频和声音发送接口，可以发送未压缩的音频及视频信号。
- **Mixer**
Mixer, 硬件混音，使用线性叠加平均的算法（如果某一路音频的音量特别小，整个混音结果音量会被拉低），混音后可设置输出的采样率。

1.4. Audio Codec 差异说明

1.4.1 Muffin 系列

Muffin 系列芯片拥有 I2S RX * 4，但 I2S RX C 和 I2S RX D 有两种模式，一种称为 share mode，一种称为 slave mode。share mode，I2S RX C 和 I2S RX A 共享 I2S Clock (Wck 以及 Bck)，I2S RX D 和 I2S RX B 共享 I2S Clock (Wck 以及 Bck)，当需要同时使用 I2S RX A 和 I2S RX C 时，两者的 I2S 参数必须完全一致，与 I2S RX C 对接的 Codec 需要将 I2S Clock 引脚 (Wck 以及 Bck) 与 I2S RX A 的 I2S Clock (Wck 以及 Bck) 相接，Data 引脚与 I2S RX C 的 Data 相接，硬件解法如下图所示。I2S RX D 与 I2S RX B 的关系也如同 I2S RX C 和 I2S RX A 一般。

slave mode, I2S RX C 和 I2S RX D 拥有独立的时钟, 但只能作为 slave 使用。这两种模式可以通过 dts sound 节点下的 i2s-rx-mode 来设置, 0 为 slave mode, 1 为 share mode。



1.4.2 Mochi 系列

Mochi 系列芯片的 audio codec 拥有如下资源: WDMA * 3, RDMA * 2, DMIC 接口(可支持 4Chn DMIC 信号) * 1, ADC(可支持 2Chn Amic/Linein) * 2, I2S-TDM RX(16 slot) * 1, DAC(可支持 2Chn Lineout) * 2, SRC * 1, HDMI TX * 1。

Mochi 系列芯片 I2S 支持最大 16slot, 采样率新增支持 96K/192K。注意: 在 I2S 的使用场景中, I2S BCK 最大不能超过 30MHz(如: $BCK=16\text{bit} \times 16\text{slots} \times 192\text{K}=49.152\text{M}$ / $BCK=32\text{bit} \times 16\text{slots} \times 192\text{K}=98.304\text{M}$, $BCK > 30\text{MHz}$ 的使用情况不支持)。

支持两个 AO 设备混音, 在 Audio Codec 框图中用 Mixer 表示。混音后可设置输出采样率到 8K/16K/32K/48K (暂不支持)。

注: Mochi 系列芯片 Audio Codec 框图中 ADC_AB_PRE 表示未经过 DPGA 前的音频信号, 如图 1-2。

1.4.3 Maruko 系列

Maruko 系列芯片, 虽然有两组 I2S RX, 但两组 I2S RX 无法同时使用。以下是两组 I2S RX 的使用场景:

I2S RX:

- 外接 codec 传输的为非 I2S 信号, 需要对 Wck、Bck 做调整, 比如接收 PCM 信号。
- 需要与 Dmic share clock, 以达到 Dmic 和 I2S RX 同步。
- 外接 codec 工作在 master mode。

I2S TRX:

- 需要使用 I2S TX。

两组 I2S RX 选用哪一组通过 dts sound 节点下的 i2s-pcm 来选择, 1 为 I2S RX, 0 为 I2S TRX。另外 I2S RX 是否与 Dmic share clock 通过 dmic-bck-share 来进行配置, 1 为 share clock, 0 为独立 clock。

1.5. API 关键字说明

- Device (音频输出设备)

AO 的 Device 指的是 audio codec 的 RDMA。Device 是 audio3.0 对 audio codec 中 DMA 的抽象, audio codec 中的 RDMA 和 AO Device 为一一对应的关系。如 AO Device0 对应 audio codec 中的 RDMA1, AO Device1 对应 audio codec 中的 RDMA2, 以此类推。audio3.0 的数据流均以 DMA 为中心进行串接。

audio3.0 中还包括另一类 Device, 即 AI 设备不经过 DMA, 直接将数据流送至 AO 设备, 如 (Amic-->Lineout)。这一类 device 称为直通通路 Device, 即 Passthrough, 目前仅支持 MI_AUDIO_PASSTHROUGH_DEV_1。

- Interface (音频输出外设)

AO 的 Interface 指的是 audio codec 的音频输出外设接口的抽象，如 Speaker/I2S Codec/HDMI 等接口。

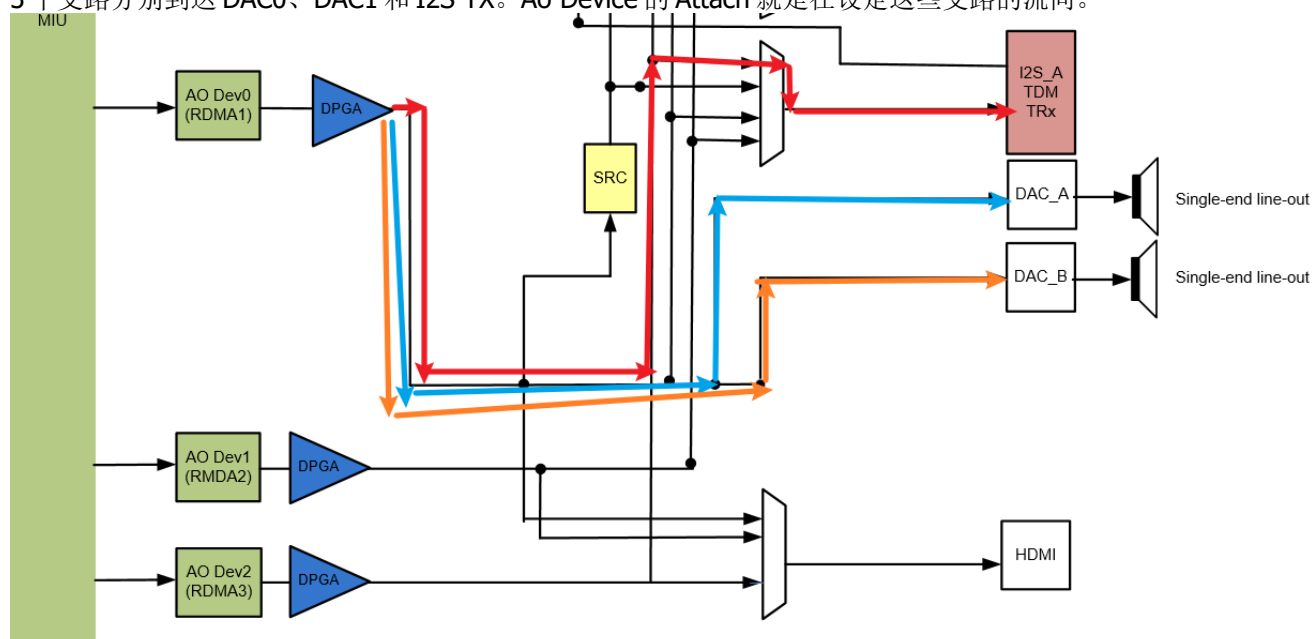
- Attach

AO 的 attach 指的是将 Interface 挂载到 RDMA 上。对于 AO Device 而言，Attach 是将 RDMA 的输出信号连接到具体的 Interface，使外设输出音频信号。AO Device 支持动态 Attach。

当同一个 AO Interface 被 Attach 到两个 AO Device 时，AO Interface 的输出为两个 AO Device 输出混音后的结果（Mochi 系列支持）。

当使用场景中包含 Passthrough 时，需要先要 attach AI，再 attach AO，否则将不能正常使用

下图是将 I2S TX 和 DAC0/1 Attach 到 RDMA1 的结果，从 RDMA1 输出的信号经过 DPGA 做增益调节后，分成 3 个支路分别到达 DAC0、DAC1 和 I2S TX。Ao Device 的 Attach 就是在设定这些支路的流向。



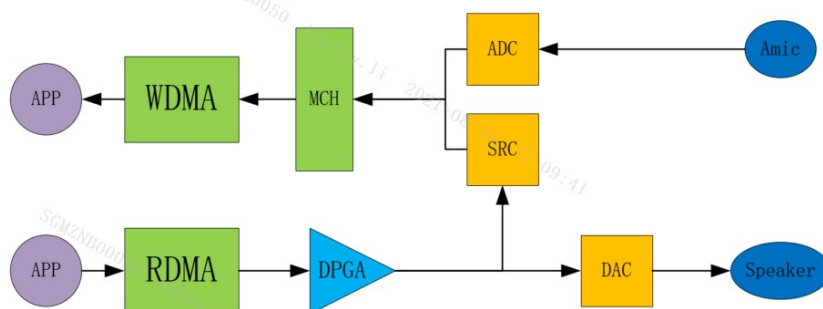
- Detach

AO 的 detach 指的是将 Interface 和 RDMA 的连接断开。

- Echo (回声)

AO 的 Echo 指的是 AEC 的参考数据。由 audio codec 框图可以看出，SRC 的输入是 RDMA 输出并经过 DPGA 放大的信号，SRC 的输出则是对输入进行重采样后的信号，可通过 Multi Channel 送进 WDMA，作为 AEC 算法的回声参考数据。

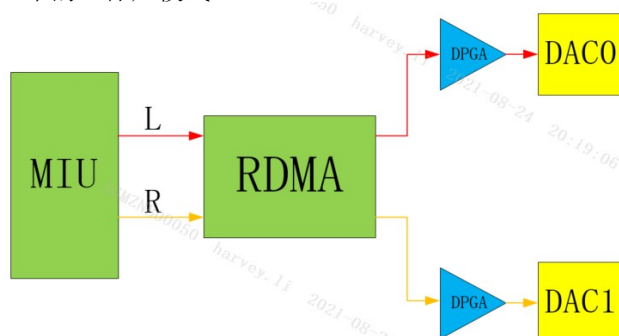
对于 AI Device 而言，Echo 表示 audio codec 框图中 SRC 的输出信号。而对于 AO Device 而言，Echo 表示将 AO Device 的输出连接到 SRC 的输入。下图以简单框图表示 AI Device 和 AO Device 同时使用 Echo 的数据流情况，应用即可获取到已经对齐的 AEC far end 和 near end 的数据。



- **Sound Mode (声音模式)**
AO 的 Sound Mode 指的是音频的声道数，如单声道、立体声。
- **Channel Mode (声道输出模式)**
支持静态设置和动态设置 Channel Mode，Muffin 系列芯片不支持动态 Channel Mode 设定。
AO 的 Channel Mode 指代的是 RDMA 通道的输出模式。对于 AO Device 而言，Channel Mode 决定了音频数据的声道和 Interface 输出左右声道的对应关系。下面结合图示来说明 Channel Mode 的作用。

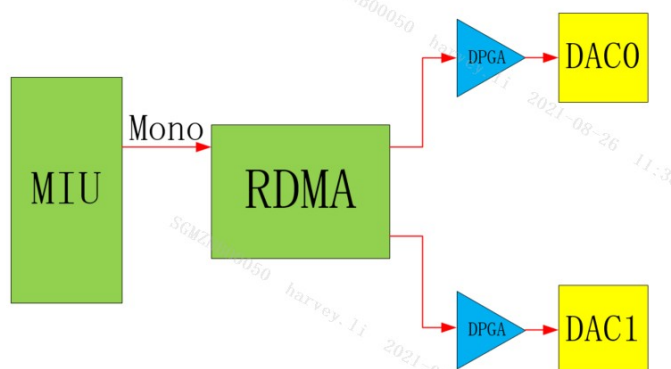
1. E_MI_AO_CHANNEL_MODE_STEREO

正常的立体声模式。

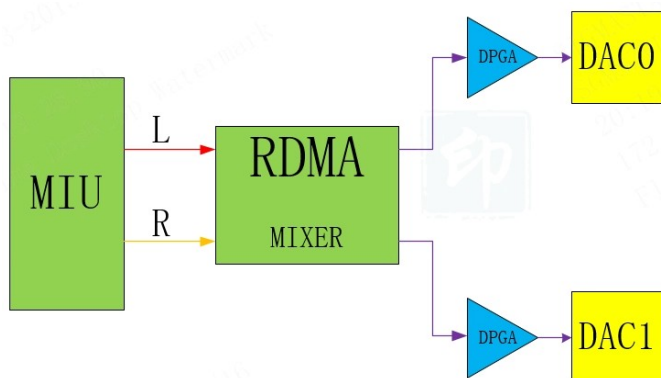


2. E_MI_AO_CHANNEL_MODE_DOUBLE_MONO

Double Mono 单声道，左右两个声道输出为同样的单声道数据。

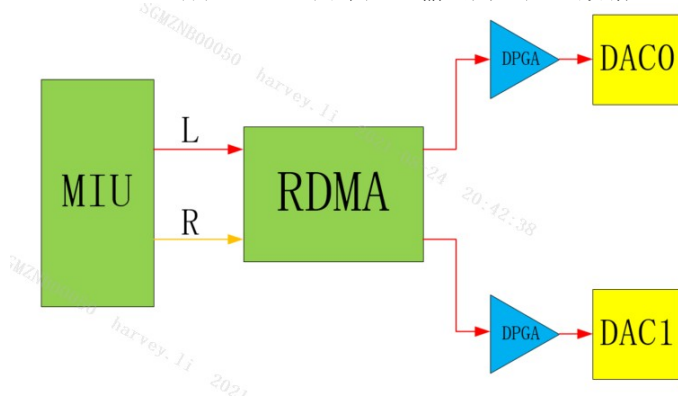


Double Mono 立体声，左右两个声道输出为左右声道混音后的数据，左右声道输出相同（Muffin 系列不支持）。



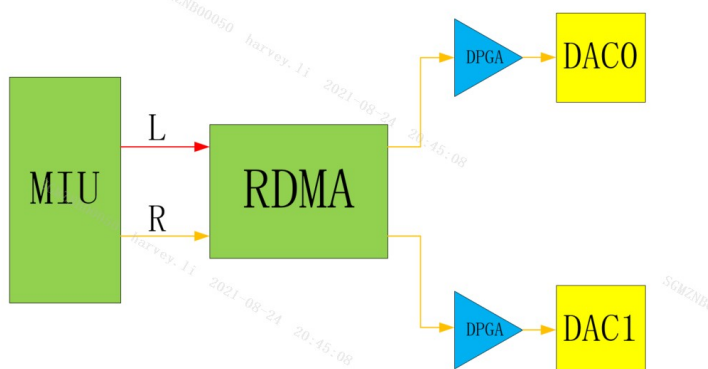
3. E_MI_AO_CHANNEL_MODE_DOUBLE_LEFT

Double Left 立体声，左右两个声道输出为左声道数据（Muffin 系列不支持）。



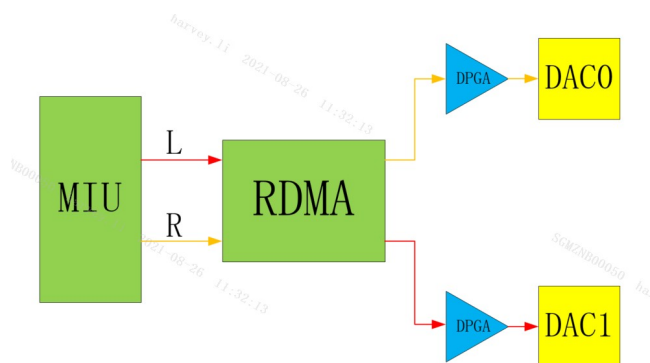
4. E_MI_AO_CHANNEL_MODE_DOUBLE_RIGHT

Double Right 立体声，左右两个声道输出为右声道数据（Muffin 系列不支持）。



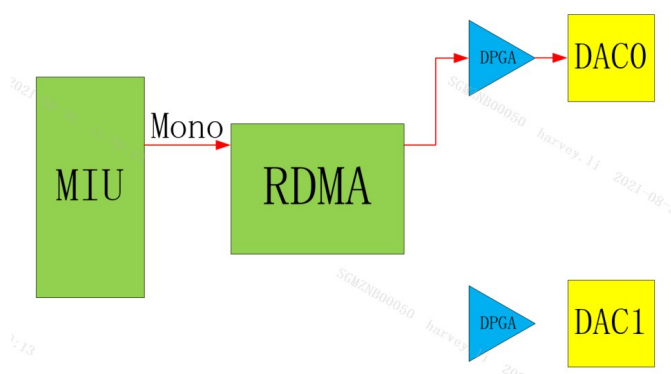
5. E_MI_AO_CHANNEL_MODE_EXCHANGE

Exchange 立体声，左右两个声道输出为左右声道互换数据。

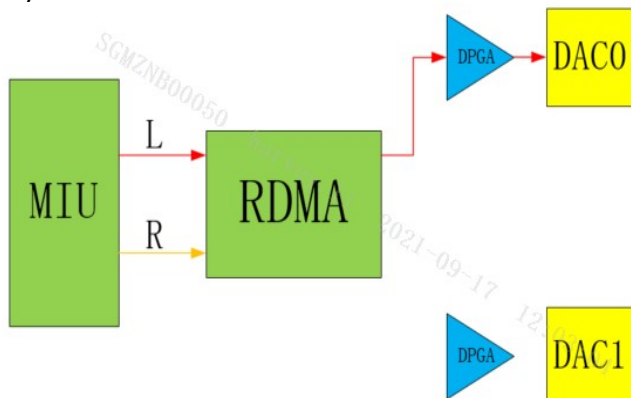


6. E_MI_AO_CHANNEL_MODE_ONLY_LEFT

Only Left 单声道，只有左声道输出为单声道数据。

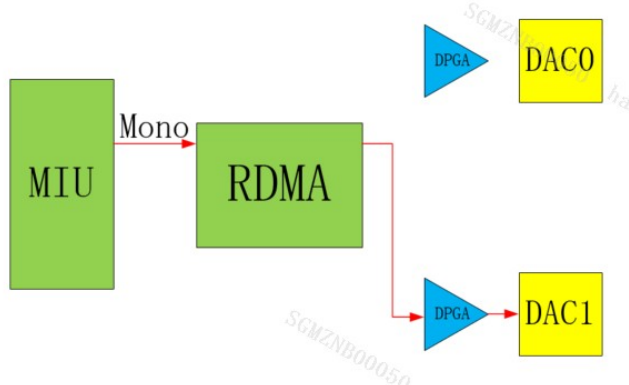


Only Left 立体声，只有左声道输出为立体声数据。

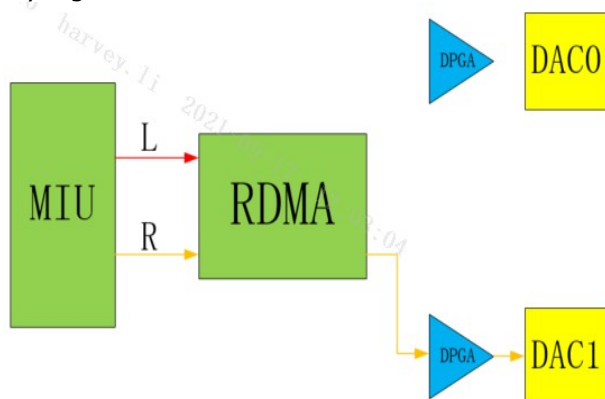


7. E_MI_AO_CHANNEL_MODE_ONLY_RIGHT

Only Right 单声道，只有右声道输出为单声道数据。



Only Right 立体声，只有右声道输出为立体声数据。



- **Gain**
AO 的 Gain 在 audio3.0 架构上分成两类，一类是与 Device 相关联的 DPGA Gain，即 audio codec 框图中的 DPGA，另一类则是 Interface 所独有的 Gain。对于 AO 而言，目前仅能通过 Dpga 调整 Gain。
- **Format**
Format，即用什么数据形式来表示一个音频采样样本，目前仅支持 S16_LE 格式(PCM Linear 16bit (Little Endian))。
- **Sample Rate**
Sample Rate，即播放采样率。
- **Period Size**
对于 AO Device 而言，Period Size 表示 AO Device 默认的起播条件(缓存中的样本数大于 Period Size，才会起播)。
- **I2S 参数**
 1. **I2S Mode**
I2S Mode 决定了 I2S 的工作模式，是标准的 I2S 模式还是 Tdm I2S 模式(2Channel 或多 channel)，是 Master 还是 Slave(Master 提供同步时钟，Slave 接收同步时钟)。一般来说，工作模式没有什么限制，能与外接的 Codec 时钟匹配上即可。

2. I2S BitWidth

I2S 收发数据的位宽，目前可支持 16/32bit，但硬件只能处理 16bit，意味着位宽为 32bit 时，低 16bit 为无效数据。

3. I2S Format

I2S 的 Format 即 I2S 的对齐方式，目前仅支持 I2S Philips 以及 Left-justified 对齐。下图分别为这两种格式的波形图。

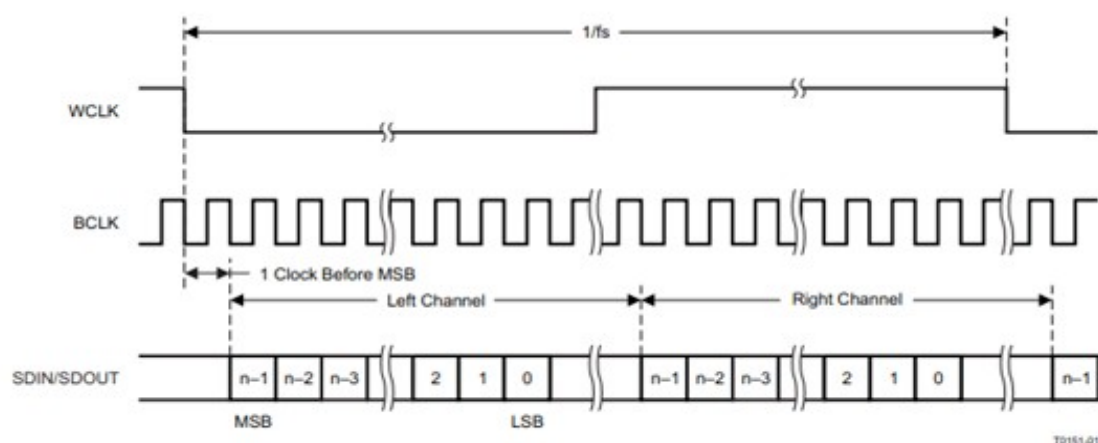


Figure 20. I²S Serial Data Bus Mode Operation

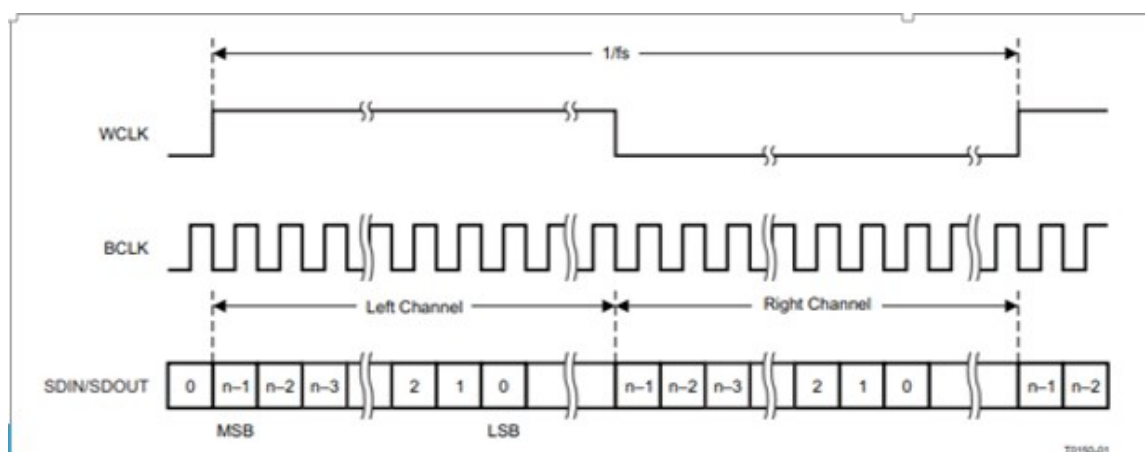


Figure 19. Left-Justified Serial Data Bus Mode Operation

I2S Philips 的对齐格式，样本数据的第一个数据位出现在 WCLK(即左右通道切换时钟)跳变的第一个 BCLK(即串行时钟)后。左对齐格式，样本数据的第一个数据位出现在 WCLK(即左右通道切换时钟)跳变的第一个 BCLK(即串行时钟)内，且 WCLK 的极性与 I2S Philips 的对齐格式相反。

4. I2S Sample Rate

I2S 收发的采样频率。

5. Mclk

Mclk, 称为主时钟, 也叫系统时钟 (System Clock), 一般是采样频率的 256 或 384 倍, 作用是为了使系统间能够更好的同步, 并不是必须的。目前仅支持 12.288M, 16.384M, 18.432M, 24.576M, 24M, 48M 等。

6. bSyncClock/4-Wire/6-Wire Mode

SigmaStar 的 I2S 有两种接线方式。一种是 4-Wire 模式, 包括 RX_WCK、RX_BCK、RX_SDI、TX_SDO 四根线, 此模式下, TX 没有独立的 clock, 所有的 clock 均由 RX 提供, 故在此模式下 TX 需要依赖 RX 来使用, 没法单独使用 TX, 且 I2S TX 的参数需与 I2S RX 一致。另一种是 6-Wire 模式, 包括 RX_WCK、RX_BCK、RX_SDI、TX_WCK、TX_BCK、TX_SDO 六根线, 此模式下 RX 和 TX 各自独立, 没有关联。

4-Wire/6-Wire Mode 的选择需要根据具体场景来决定。

MI API I2S 参数中的 bSyncClock 为 TRUE, 则使用 4-Wire Mode, FALSE 则使用 6-Wire Mode, 且属于同一组 I2S 的 RX 和 TX 不能一边设置成 4-Wire Mode, 另一边设置成 6-Wire Mode。

7. Slot

Slot 表示 I2S 传输的声道数, 目前 I2S 模式下支持 2slot, Tdm 模式下支持 4/8/16slot, 但 I2S TX 的有效数据为 2slot, 当对 I2S TX 设置大于 2 的 slot 个数时, 除了 slot0 和 slot1, 其他的均为无效数据。

● Passthrough(直通通道)

Passthrough 意指 AI 设备经直接将数据流送至 AO 的设备, 不经 DMA, 以 ADC_AB-->DAC_AB 为例, 如图 1-3 红色标注的通路:

Muffin 系列芯片仅支持输入设备为 ADC_AB 到输出设备为 DAC_AB 的 Passthrough。

Mochi 系列芯片支持输入设备为 ADC_AB/DMIC_A_01/ DMIC_A_23/I2S_A_01/.../ I2S_A_EF 到输出设备为 DAC_AB/I2S_TX/ HDMI 的任意组合 Passthrough。Mochi 系列芯片目前 Passthrough 的限制为仅支持采样率 48K; Passthrough Device 暂不支持 Channel Mode 设定; Passthrough + DMA 混音场景时, 输出设备仅支持 DAC_AB。

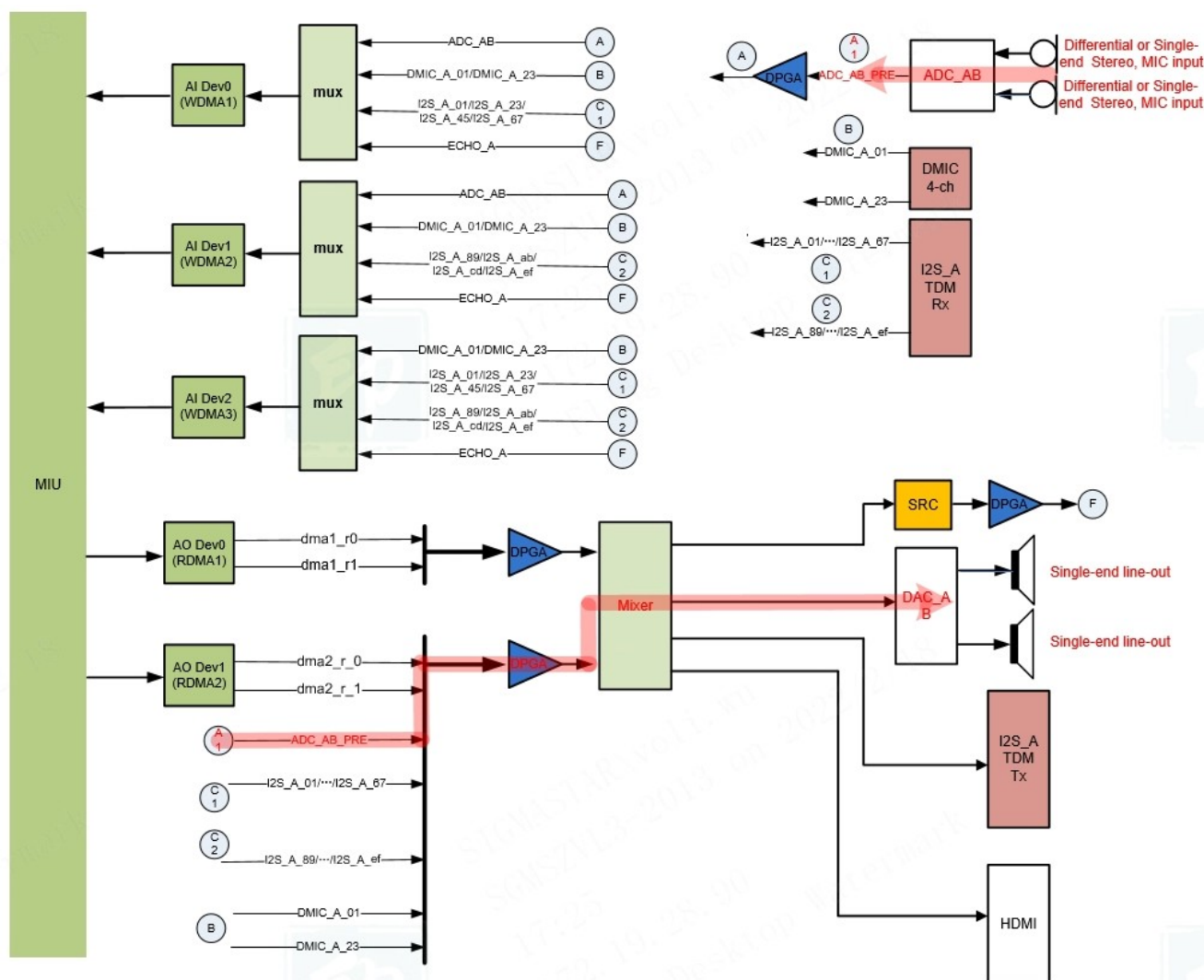


图 1-3: Mochi 系列芯片直通通路示意图

2. API 参考

音频输出（AO）主要用于启用音频输出设备、发送音频帧到输出通道等。

API 名	功能
MI_AO_Open	打开音频输出设备
MI_AO_OpenWithCfgFile	打开音频输出设备，并按配置文件初始化
MI_AO_Close	关闭音频输出设备
MI_AO_AttachIf	挂载外设到音频输出设备
MI_AO_DetachIf	卸载外设与音频输出设备的连接
MI_AO_Write	写入音频数据
MI_AO_Start	启动音频输出设备，立刻进行播放
MI_AO_Stop	停止音频输出设备，立刻停止播放
MI_AO_Pause	暂停音频输出设备
MI_AO_Resume	回复音频输出设备
MI_AO_SetVolume	设置音频输出设备的音量
MI_AO_GetVolume	获取音频输出设备的音量
MI_AO_SetMute	设置音频输出设备的静音参数
MI_AO_GetMute	获取音频输出设备的静音参数
MI_AO_SetIfVolume	设置音频输出外设的音量
MI_AO_GetIfVolume	获取音频输出外设的音量
MI_AO_SetIfMute	设置音频输出外设的静音参数
MI_AO_GetIfMute	获取音频输出外设的静音参数
MI_AO_SetI2SConfig	设置 I2S TX 的配置参数
MI_AO_GetI2SConfig	获取 I2S TX 的配置参数
MI_AO_AdjustSpeed	调整音频输出设备的播放速度
MI_AO_GetTimestamp	获取音频输出设备当前播放时间戳和缓存数据量
MI_AO_GetLatency	获取音频输出设备的延迟
MI_AO_InitDev	初始化音频输出设备
MI_AO_DeinitDev	反初始化音频输出设备
MI_AO_GetAttr	获取音频输出设备属性
MI_AO_SetChannelMode	动态设置声道输出模式（Muffin 系列不支持）

2.1. MI_AO_Open

➤ 功能

打开音频输出设备。

➤ 语法

```
MI_S32 MI_AO_Open(MI_AUDIO_DEV AoDevId, const MI_AO_Attr_t *pstAttr);
```

➤ 形参

参数名称	描述	输入/输出
AoDevId	音频输出设备号	输入
pstAttr	音频输出设备属性指针	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_ao.h
- 库文件：libmi_ao.a/libmi_ao.so

※ 注意

音频输出设备属性包括数据格式、声音模式、采样率、起播条件的样本数、声道输出模式。

- 音频数据格式([MI_AUDIO_Format_e](#))
音频数据格式指采样样本的数据格式。目前仅支持 S16_LE。
- 音频声音模式([MI_AUDIO_SoundMode_e](#))
音频声音模式指待播放数据的声道数，单声道/立体声。
- 音频采样率([MI_AUDIO_SampleRate_e](#))
音频采样率指一秒内的采样次数，采样率越高表明失真度越小，处理的数据量也就随之增加。一般来说语音使用 8k 采样率，音频使用 32k 或以上的采样率。
- 起播条件的样本数([u32PeriodSize](#))
起播条件即数据缓存满足此样本数才会开始播放。此参数设定后，建议应用按此大小来写入音频数据。当起播条件的样本数为 0 时，APP 需要主动调用 MI_AO_Start 来启动播放。
- 声道输出模式([MI_AO_ChannelMode_e](#))
声道输出模式决定了音频数据在音频设备的输出模式。
- Maruko 系列芯片 I2S TX 的行为不受 [MI_AO_ChannelMode_e](#) 参数的影响。

➤ 举例

简单例子如下：

```
1. MI_AO_Attr_t stAoSetAttr;  
2. memset(&stAoSetAttr, 0x0, sizeof(MI_AO_Attr_t));  
3. stAoSetAttr.enFormat = E_MI_AUDIO_FORMAT_PCM_S16_LE;
```

```
4. stAoSetAttr.enSoundMode = E_MI_AUDIO_SOUND_MODE_MONO;
5. stAoSetAttr.enSampleRate = E_MI_AUDIO_SAMPLE_RATE_8000;
6. stAoSetAttr.u32PeriodSize = 1024;
7. stAoSetAttr.enChannelMode = E_MI_AO_CHANNEL_MODE_DOUBLE_MONO;
8. ExecFunc(MI_AO_Open(AoDevId, &stAoSetAttr), MI_SUCCESS);
```

详细例子如下:

```
1. MI_AUDIO_DEV AoDevId = 0;
2. MI_AO_Attr_t stAoSetAttr, stAoGetAttr;
3. MI_S8 s8LeftVolume, s8RightVolume;
4. MI_AO_GainFading_e eGainFading;
5.
6. memset(&stAoSetAttr, 0x0, sizeof(MI_AO_Attr_t));
7.
8. // 设置 AO Device 的 Format 为 S16_LE
9. stAoSetAttr.enFormat = E_MI_AUDIO_FORMAT_PCM_S16_LE;
10.
11. // 设置 AO Device 的 Sound Mode 为 Mono
12. stAoSetAttr.enSoundMode = E_MI_AUDIO_SOUND_MODE_MONO;
13.
14. // 设置 AO Device 的采样率为 8KHz
15. stAoSetAttr.enSampleRate = E_MI_AUDIO_SAMPLE_RATE_8000;
16.
17. // 设置 AO Device 的起播条件为 1024 个采样样本
18. stAoSetAttr.u32PeriodSize = 1024;
19.
20. // 设置 AO Device 的 Channel Mode 为 Double Mono
21. stAoSetAttr.enChannelMode = E_MI_AO_CHANNEL_MODE_DOUBLE_MONO;
22.
23. // 打开 AO Device
24. ExecFunc(MI_AO_Open(AoDevId, &stAoSetAttr), MI_SUCCESS);
25.
26. // 获取 AO Device 属性
27. ExecFunc(MI_AO_GetAttr(AoDevId, &stAoGetAttr), MI_SUCCESS);
28.
29. // 将 DAC_AB attach 到 AO Device
30. ExecFunc(MI_AO_AttachIf(AoDevId, E_MI_AO_IF_DAC_AB, 0), MI_SUCCESS);
31.
32. // 动态设置音频通道模式为 Only Left
33. ExecFunc(MI_AO_SetChannelMode(AoDevId, E_MI_AO_CHANNEL_MODE_ONLY_LEFT), MI_SUCCESS);
34.
35. // 设置 Dpga Gain
36. s8LeftVolume = 0;
37. s8RightVolume = 0;
38. eGainFading = E_MI_AO_GAIN_FADING_OFF;
39. ExecFunc(MI_AO_SetVolume(AoDevId, s8LeftVolume, s8RightVolume, eGainFading), MI_SUCCESS);
40.
41. // 往 AO Device 写数据
42. MI_AO_Write(AoDevId, u8TempBuf, s32ReadSize, 0, -1);
43.
44. // detach DAC_AB
45. ExecFunc(MI_AO_DetachIf(AoDevId, E_MI_AO_IF_DAC_AB), MI_SUCCESS);
46.
47. // 关闭 AO Device
48. ExecFunc(MI_AO_Close(MI_AO_DEV_1), MI_SUCCESS);
```

2.2. MI_AO_OpenWithCfgFile

➤ 功能

打开音频输出设备，并按配置文件初始化。

➤ 语法

```
MI_S32 MI_AO_OpenWithCfgFile(MI\_AUDIO\_DEV AoDevId, const char *pCfgPath);
```

➤ 形参

参数名称	描述	输入/输出
AoDevId	音频输出设备号	输入
pCfgPath	配置文件路径	输入

➤ 返回值

返回值 $\left\{ \begin{array}{l} 0 \quad \text{成功。} \\ \text{非} 0 \quad \text{失败，参照[错误码](#)。} \end{array} \right.$

➤ 依赖

- 头文件：mi_ao.h
- 库文件：libmi_ao.a/libmi_ao.so

※ 注意

- 此接口相当于 [MI_AO_Open](#) 和 [MI_AO_AttachIf](#) 组合。
- 如果音频输出设备已经处于开启状态，则直接返回成功。
- 配置文件模板如下：

```
1. ; Device attr section
2. [DEV]
3. ; enFormat determines the data format of the sample, only supports S16_LE now.
4. ; 0[S16_LE]
5. enFormat = 0
6. ; enSoundMode determines the sound mode of AO Device.
7. ; 1[Mono] 2[Stereo]
8. enSoundMode = 2
9. ; enSampleRate determines the sample rate of AO Device.
10. ; 8000[8KHz] 16000[16KHz] 32000[32KHz] 48000[48KHz]
11. enSampleRate = 8000
12. ; u32PeriodSize(samples) determines the time to start RDMA.
13. u32PeriodSize = 1024
14. ; enChannelMode determines the channel mode of AO Device.
15. ; 0[Stereo] 1[Double Mono] 2[Double Left] 3[Double Right] 4[Exchange] 5[Only Left] 6[Only Right]
16. enChannelMode = 0
17. ; attach interface
18. ; 1[DAC_AB] 2[DAC_CD] 4[I2S_A] 8[I2S_B] 16[ECHO_A] 32[HDMI_A]
19. ; if you want to attach DAC_AB and DAC_CD, so enAoIfs = DAC_AB | DAC_CD = 1 | 2 = 3.
20. enAoIfs = 5
21.
22. ; I2S TX attr section
23. ; It is not necessary when you're not using I2S TX
24. [I2S_A]
```

```

25. ; enMode determines the working mode of I2S Tx
26. ; 0[I2S Master] 1[I2S Slave] 2[Tdm Master] 3[Tdm Slave]
27. enMode = 0
28. ; enBitWidth determines the bit width of I2S Tx
29. ; 0[16 bit] 1[32bit]
30. enBitWidth = 0
31. ; enFormat determines the waveform alignment of I2S Tx
32. ; 0[I2S Philips] 1[I2S Left-justify]
33. enFormat = 0
34. ; enSampleRate determines the sample rate of I2S Tx
35. ; 8000[8KHz] 16000[16KHz] 32000[32KHz] 48000[48KHz]
36. enSampleRate = 8000
37. ; enMclk determines the frequency of Mclk
38. ; 0[disable Mclk] 1[12.288M] 2[16.384M] 3[18.432M] 4[24.576M] 5[24M] 6[48M]
39. enMclk = 0
40. ; bSyncClock: 0[4-wire mode] 1[6-wire mode]
41. bSyncClock = 0
42. ; u32TdmSlots determines the slot number of I2S Tx
43. u32TdmSlots = 2

```

上面的配置信息，enFormat = 0 表示 AO Device 采用 S16_LE 格式，enSoundMode = 1 表示 Sound Mode 采用 Mono，enSampleRate = 8000 表示 AO Device 使用 8KHz 采样频率，u32PeriodSize = 1024 表示起播条件为 1024 个采样点，enChannelMode = 1 表示 Double Mono，enAoIfs=1 表示将 DAC0/1 attach 到 RDMA。若需要使用 I2S TX 还需要设置 I2S TX 的参数，上述配置作用于 I2S_A，enMode = 0 表示使用 I2S Master 模式，enBitWidth = 0 表示 I2S 的接收位宽为 16bit，enFormat = 0 表示按 I2S Philips 方式对齐，enSampleRate = 8000 表示 I2S RX 的采样频率为 8KHz，enMclk = 0 表示不使用 Mclk，bSyncClock = 0 表示 4-wire 模式，u32TdmSlots 表示接收 2 通道数据。注意使用时需把注释行删掉。



AO template.ini

➤ 举例

```

1. char *path = "/tmp/Dev0Cfg.ini";
2. ExecFunc(MI_AO_OpenWithCfgFile(AoDevId, path), MI_SUCCESS);

```

2.3. MI_AO_Close

➤ 功能

关闭音频输出设备。

➤ 语法

```
MI_S32 MI_AO_Close(MI_AUDIO_DEV AoDevId);
```

➤ 形参

参数名称	描述	输入/输出
AoDevId	音频输出设备号	输入

➤ 返回值

返回值 $\begin{cases} 0 & \text{成功。} \\ \text{非 } 0 & \text{失败，参照[错误码](#)。} \end{cases}$

➤ 依赖

- 头文件：mi_ao.h
- 库文件：libmi_ao.a/libmi_ao.so

※ 注意

- 如果音频输出设备已经处于关闭状态，则直接返回成功。

➤ 举例

简单例子如下：

1. ExecFunc(MI_AO_Close(MI_AO_DEV_1), MI_SUCCESS);

详细例子请参考 [MI_AO_Open](#)。

2.4. MI_AO_AttachIf

➤ 功能

挂载外设到音频输出设备。

➤ 语法

MI_S32 MI_AO_AttachIf([MI_AUDIO_DEV](#) AoDevId, [MI_AO_If_e](#) enAoIfs, MI_U32 u32AudioDelay);

➤ 形参

参数名称	描述	输入/输出
AoDevId	音频输出设备号	输入
enAoIfs	AO Interface, 需要挂载到音频输出设备的外设信息，可以使用 “ ” 来表示挂载多个外设	输入
u32AudioDelay	延迟参数，保留未使用	输入

➤ 返回值

返回值 $\begin{cases} 0 & \text{成功。} \\ \text{非 } 0 & \text{失败，参照[错误码](#)。} \end{cases}$

➤ 依赖

- 头文件：mi_ao.h
- 库文件：libmi_ao.a/libmi_ao.so

※ 注意

- 此接口仅能在 [MI_AO_Open](#) 成功后调用。
- 如需要挂载 I2S TX 到音频输出设备，需要先调用 [MI_AO_SetI2SConfig](#) 来初始化 I2S TX。

- 如果要获取回声参考数据，AO 模块需要 attach E_MI_AO_IF_ECHO_A，AI 模块需要 attach E_MI_AI_IF_ECHO_A。

➤ 举例

```
1. ExecFunc(MI_AO_AttachIf(AoDevId, E_MI_AO_IF_DAC_AB, 0), MI_SUCCESS);
```

2.5. MI_AO_DetachIf

➤ 功能

卸载外设与音频输出设备的连接。

➤ 语法

MI_S32 MI_AO_DetachIf([MI_AUDIO_DEV](#) AoDevId, [MI_AO_If_e](#) enAoIfs);

➤ 形参

参数名称	描述	输入/输出
AoDevId	音频输出设备号	输入
enAoIfs	AO Interface, 需要卸载到音频输出设备的外设信息, 可以使用 “ ” 来表示卸载多个外设	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败, 参照[错误码](#)。

➤ 依赖

- 头文件: mi_ao.h
- 库文件: libmi_ao.a/libmi_ao.so

※ 注意

- 此接口仅能在 [MI_AO_AttachIf](#) 成功后调用。
- Maruko 系列芯片 I2S TX 和 ECHO 不支持 detach。

➤ 举例

简单例子如下:

```
1. ExecFunc(MI_AO_DetachIf(AoDevId, E_MI_AO_IF_DAC_AB), MI_SUCCESS);
```

详细例子请参考 [MI_AO_Open](#)。

2.6. MI_AO_Write

➤ 功能

写入音频数据。

➤ 语法

MI_S32 MI_AO_Write([MI_AUDIO_DEV](#) AoDevId, const void *pvBuffer, MI_U32 u32Bytes, MI_U64 u64Pts, MI_S32 s32TimeoutMs);

➤ 形参

参数名称	描述	输入/输出
AoDevId	音频输出设备号	输入
pvBuffer	音频数据指针	输入
u32Bytes	音频数据长度	输入
u64Pts	音频数据时间戳，保留未使用	输入
s32TimeoutMs	写入音频数据的超时时间 -1 表示阻塞模式，缓存空间不足时一直等待 0 表示非阻塞模式，缓存空间不足时则报错返回 >0表示阻塞s32TimeoutMs毫秒，超时则报错返回	输入

➤ 返回值

返回值 $\begin{cases} 0 & \text{成功。} \\ \text{非 } 0 & \text{失败，参照[错误码](#)。} \end{cases}$

➤ 依赖

- 头文件：mi_ao.h
- 库文件：libmi_ao.a/libmi_ao.so

※ 注意

- 此接口仅能在 [MI_AO_Open](#) 和 [MI_AO_AttachIf](#) 成功后调用。
- s32TimeoutMs 必须大于等于-1，等于-1 时采用阻塞模式写入数据，等于 0 时采用非阻塞模式写入数据，大于 0 时，阻塞 s32MilliSec 毫秒后，仍然写入不成功则返回超时并报错。

➤ 举例

简单例子如下：

```
1. MI_U8 u8TempBuf[1024] = {0};
2. MI_S32 s32ReadSize;
3. s32ReadSize = read(s32Fd, pu8TempBuf, sizeof(u8TempBuf));
4. if (s32ReadSize > 0)
5. {
6.     MI_AO_Write(AoDevId, u8TempBuf, s32ReadSize, 0, -1);
7. }
```

详细例子请参考 [MI_AO_Open](#)。

2.7. MI_AO_Start

➤ 功能

启动音频输出设备，立刻进行播放。

➤ 语法

```
MI_S32 MI_AO_Start(MI\_AUDIO\_DEV AoDevId);
```

➤ 形参

参数名称	描述	输入/输出
AoDevId	音频输出设备号	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_ao.h
- 库文件：libmi_ao.a/libmi_ao.so

※ 注意

- 此接口仅能在 [MI_AO_Open](#) 成功后调用。
- 当此接口被调用时，即使没有达到默认的起播条件(> u32PeriodSize)，也会立刻进行播放。

➤ 举例

```
1. ExecFunc(MI_AO_Start(AoDevId),MI_SUCCESS);
```

2.8. MI_AO_Stop

➤ 功能

停止音频输出设备，立刻停止播放。

➤ 语法

```
MI_S32 MI_AO_Stop(MI\_AUDIO\_DEV AoDevId);
```

➤ 形参

参数名称	描述	输入/输出
AoDevId	音频输出设备号	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件: `mi_ao.h`
- 库文件: `libmi_ao.a/libmi_ao.so`

※ 注意

- 此接口仅能在 [MI_AO_Open](#) 成功后调用。
- 当此接口被调用时, 会立刻停止播放。停止后原有的缓存数据也会丢失。

➤ 举例

```
1. ExecFunc(MI_AO_Stop(AoDevId),MI_SUCCESS);
```

2.9. MI_AO_Pause

➤ 功能

暂停音频输出设备。

➤ 语法

MI_S32 MI_AO_Pause ([MI_AUDIO_DEV](#) AoDevId);

➤ 形参

参数名称	描述	输入/输出
AoDevId	音频输出设备号	输入

➤ 返回值

返回值 $\begin{cases} 0 & \text{成功。} \\ \text{非 } 0 & \text{失败, 参照[错误码](#)。} \end{cases}$

➤ 依赖

- 头文件: `mi_ao.h`
- 库文件: `libmi_ao.a/libmi_ao.so`

※ 注意

- 此接口仅能在 [MI_AO_Open](#) 成功后调用。
- 当此接口被调用时, 会立刻暂停播放。

➤ 举例

```
1. ExecFunc(MI_AO_Pause(AoDevId),MI_SUCCESS);
```

2.10. MI_AO_Resume

➤ 功能

恢复音频输出设备。

➤ 语法

```
MI_S32 MI_AO_Resume(MI\_AUDIO\_DEV AoDevId);
```

➤ 形参

参数名称	描述	输入/输出
AoDevId	音频输出设备号	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_ao.h
- 库文件：libmi_ao.a/libmi_ao.so

※ 注意

- 此接口仅能在 [MI_AO_Pause](#) 成功后调用。
- 当此接口被调用时，会立刻恢复播放。

➤ 举例

```
1. ExecFunc(MI_AO_Resume(AoDevId),MI_SUCCESS);
```

2.11. MI_AO_SetVolume

➤ 功能

设置音频输出设备的音量。

➤ 语法

```
MI_AO_SetVolume(MI\_AUDIO\_DEV AoDevId, MI_S8 s8LeftVolume, MI_S8 s8RightVolume,  
MI\_AO\_GainFading\_e enFading);
```

➤ 形参

参数名称	描述	输入/输出
AoDevId	音频输出设备号	输入
s8LeftVolume	左声道音量(-60 ~ 30dB, 1dB/step)	输入
s8RightVolume	右声道音量(-60 ~ 30dB, 1dB/step)	输入
eFading	音频增益变化的速度	输入

➤ 返回值

{ 0 成功。

返回值

非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_ao.h
- 库文件：libmi_ao.a/libmi_ao.so

※ 注意

- 此接口仅能在 [MI_AO_Open](#) 成功后调用。

➤ 举例

```
1. MI_S8 s8LeftVolume, s8RightVolume;
2. MI_AO_GainFading_e eGainFading;
3. s8LeftVolume = 0;
4. s8RightVolume = 0;
5. eGainFading = E_MI_AO_GAIN_FADING_OFF;
6. ExecFunc(MI_AO_SetVolume(AoDevId, s8LeftVolume, s8RightVolume, eGainFading), MI_SUCCESS);
```

2.12. MI_AO_GetVolume

➤ 功能

获取音频输出设备的音量。

➤ 语法

MI_S32 MI_AO_GetVolume([MI_AUDIO_DEV](#) AoDevId, MI_S8 *ps8LeftVolume, MI_S8 *ps8RightVolume);

➤ 形参

参数名称	描述	输入/输出
AoDevId	音频输出设备号	输入
ps8LeftVolume	左声道音量指针	输出
ps8RightVolume	右声道音量指针	输出

➤ 返回值

返回值 $\left\{ \begin{array}{ll} 0 & \text{成功。} \\ \text{非 } 0 & \text{失败，参照[错误码](#)。} \end{array} \right.$

➤ 依赖

- 头文件：mi_ao.h
- 库文件：libmi_ao.a/libmi_ao.so

※ 注意

- 此接口仅能在 [MI_AO_SetVolume](#) 成功后调用。

➤ 举例

```
1. MI_S8 s8LeftVolume, s8RightVolume;
2. ExecFunc(MI_AO_GetVolume(AoDevId, &s8LeftVolume, &s8RightVolume), MI_SUCCESS);
```

2.13. MI_AO_SetMute

➤ 功能

设置音频输出设备的静音参数。

➤ 语法

MI_S32 MI_AO_SetMute([MI_AUDIO_DEV](#) AoDevId, MI_BOOL bLeftMute, MI_BOOL bRightMute, [MI_AO_GainFading_e](#) enFading);

➤ 形参

参数名称	描述	输入/输出
AoDevId	音频输出设备号	输入
bLeftMute	左声道静音参数	输入
bRightMute	右声道静音参数	输入
eFading	音频增益变化的速度，保留未使用	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_ao.h
- 库文件：libmi_ao.a/libmi_ao.so

※ 注意

- 此接口仅能在 [MI_AO_Open](#) 成功后调用。
- 当使用 [MI_AO_SetVolume](#) 调节音量，会自动退出静音状态。

➤ 举例

```
1. MI_BOOL bLeftMute = TRUE;
2. MI_BOOL bRightMute = TRUE;
3. MI_AO_GainFading_e eGainFading = E_MI_AO_GAIN_FADING_OFF;
4. ExecFunc(MI_AO_SetMute(AoDevId, bLeftMute, bRightMute, eGainFading), MI_SUCCESS);
```


2.14. MI_AO_GetMute

➤ 功能

获取音频输出设备的静音参数。

➤ 语法

```
MI_S32 MI_AO_GetMute(MI\_AUDIO\_DEV AoDevId, MI_BOOL *pbLeftMute, MI_BOOL *pbRightMute);
```

➤ 形参

参数名称	描述	输入/输出
AoDevId	音频输出设备号	输入
pbLeftMute	左声道静音参数指针	输出
pbRightMute	右声道静音参数指针	输出

➤ 返回值

返回值 $\begin{cases} 0 & \text{成功。} \\ \text{非 } 0 & \text{失败，参照[错误码](#)。} \end{cases}$

➤ 依赖

- 头文件：mi_ao.h
- 库文件：libmi_ao.a/libmi_ao.so

※ 注意

- 此接口仅能在 [MI_AO_Open](#) 成功后调用。

➤ 举例

```
1. MI_BOOL bLeftMute;
2. MI_BOOL bRightMute;
3. ExecFunc(MI_AO_GetMute(AoDevId, bLeftMute, bRightMute), MI_SUCCESS);
```

2.15. MI_AO_SetIfVolume

➤ 功能

设置音频输出外设的音量。

➤ 语法

```
MI_S32 MI_AO_SetIfVolume(MI\_AO\_If\_e enAoIf, MI_S8 s8LeftVolume, MI_S8 s8RightVolume);
```

➤ 形参

参数名称	描述	输入/输出
enAoIf	音频输出外设	输入

s8LeftVolume	左声道音量	输入
s8RightVolume	右声道音量	输入

➤ 返回值

返回值 $\begin{cases} 0 & \text{成功。} \\ \text{非 } 0 & \text{失败，参照[错误码](#)。} \end{cases}$

➤ 依赖

- 头文件：mi_ao.h
- 库文件：libmi_ao.a/libmi_ao.so

※ 注意

- 目前没有 interface 支持该功能。

➤ 举例

```
1. MI_S8 s8LeftVolume = 0;
2. MI_S8 s8RightVolume = 0;
3. ExecFunc(MI_AO_SetIfVolume(E_MI_AO_IF_DAC_AB, s8LeftVolume, s8RightVolume), MI_SUCCESS);
```

2.16. MI_AO_GetIfVolume

➤ 功能

获取音频输出外设的音量。

➤ 语法

MI_S32 MI_AO_GetIfVolume([MI_AO_If_e](#) enAoIf, MI_S8 *ps8LeftVolume, MI_S8 *ps8RightVolume);

➤ 形参

参数名称	描述	输入/输出
enAoIf	音频输出外设	输入
ps8LeftVolume	左声道音量指针	输出
ps8RightVolume	右声道音量指针	输出

➤ 返回值

返回值 $\begin{cases} 0 & \text{成功。} \\ \text{非 } 0 & \text{失败，参照[错误码](#)。} \end{cases}$

➤ 依赖

- 头文件：mi_ao.h
- 库文件：libmi_ao.a/libmi_ao.so

※ 注意
无。

➤ 举例

```
1. MI_S8 s8LeftVolume = 0;  
2. MI_S8 s8RightVolume = 0;  
3. ExecFunc(MI_AO_GetIfVolume(E_MI_AO_IF_DAC_AB, &s8LeftVolume, &s8RightVolume), MI_SUCCESS);
```

2.17. MI_AO_SetIfMute

➤ 功能

设置音频输出外设的静音参数。

➤ 语法

MI_S32 MI_AO_SetIfMute([MI_AO_If_e](#) enAoIf, MI_BOOL bLeftMute, MI_BOOL bRightMute);

➤ 形参

参数名称	描述	输入/输出
enAoIf	音频输出外设	输入
bLeftMute	左声道静音参数	输入
bRightMute	右声道静音参数	输入

➤ 返回值

返回值 $\left\{ \begin{array}{ll} 0 & \text{成功。} \\ \text{非 } 0 & \text{失败，参照[错误码](#)。} \end{array} \right.$

➤ 依赖

- 头文件：mi_ao.h
- 库文件：libmi_ao.a/libmi_ao.so

※ 注意

- 目前仅 DAC_AB 支持此接口。

➤ 举例

```
1. MI_BOOL bLeftMute = 0;  
2. MI_BOOL bRightMute = 0;  
3. ExecFunc(MI_AO_SetIfMute(E_MI_AO_IF_DAC_AB, bLeftMute, bRightMute), MI_SUCCESS);
```

2.18. MI_AO_GetIfMute

➤ 功能

获取音频输出外设的静音参数。

➤ 语法

```
MI_S32 MI_AO_GetIfMute(MI\_AO\_If\_e enAoIf, MI_BOOL *pbLeftMute, MI_BOOL *pbRightMute);
```

➤ 形参

参数名称	描述	输入/输出
enAoIf	音频输出外设	输入
pbLeftMute	左声道静音参数指针	输出
pbRightMute	右声道静音参数指针	输出

➤ 返回值

返回值 $\left\{ \begin{array}{l} 0 \quad \text{成功。} \\ \text{非 } 0 \quad \text{失败，参照[错误码](#)。} \end{array} \right.$

➤ 依赖

- 头文件: `mi_ao.h`
- 库文件: `libmi_ao.a/libmi_ao.so`

※ 注意

无。

➤ 举例

```
1. MI_BOOL bLeftMute = 0;
2. MI_BOOL bRightMute = 0;
3. ExecFunc(MI_AO_GetIfMute(E_MI_AO_IF_DAC_AB, &s8LeftVolume, &s8RightVolume), MI_SUCCESS);
```

2.19. MI_AO_SetI2SConfig

➤ 功能

设置 I2S TX 的配置信息。

➤ 语法

```
MI_S32 MI_AO_SetI2SConfig(MI\_AO\_If\_e enAoI2SIf, const MI\_AUDIO\_I2sConfig\_t *pstConfig);
```

➤ 形参

参数名称	描述	输入/输出
enAiI2SIf	音频I2S TX输出外设	输入
pstConfig	音频I2S TX的配置信息	输入

➤ 返回值

返回值 $\left\{ \begin{array}{l} 0 \quad \text{成功。} \end{array} \right.$

非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件: `mi_ao.h`
- 库文件: `libmi_ao.so/libmi_ao.a`

※ 注意

- [MI_AUDIO_I2sConfig_t](#) 中的 `enSampleRate` 必须和 [MI_AO_Attr_t](#) 中的 `enSampleRate` 一致，否则 `attach` 会报错。
- 当同一个 I2S TX interface 被所有 device detach 后，即没有 device attach 上该 I2S TX interface，会重置 I2S TX 的参数，若需要 attach 到某一个 device，必须先重新设定 I2S 参数。

➤ 举例

```
1. MI_AUDIO_I2sConfig_t stAoI2sACfg;
2. memset(&stAoI2sACfg, 0x0, sizeof(stAoI2sACfg));
3. stAoI2sACfg.enMode = E_MI_AUDIO_I2S_MODE_I2S_MASTER;
4. stAoI2sACfg.enFormat = E_MI_AUDIO_I2S_FMT_I2S_MSB;
5. stAoI2sACfg.enSampleRate = E_MI_AUDIO_SAMPLE_RATE_8000;
6. stAoI2sACfg.enMclk = E_MI_AUDIO_I2S_MCLK_0;
7. stAoI2sACfg.bSyncClock = TRUE;
8. stAoI2sACfg.u32TdmSlots = 2;
9. stAoI2sACfg.enBitWidth = E_MI_AUDIO_BIT_WIDTH_16;
10. ExecFunc(MI_AO_SetI2SConfig(E_MI_AO_IF_I2S_A, &stAoI2sACfg), MI_SUCCESS);
```

2.20. MI_AO_GetI2SConfig

➤ 功能

获取 I2S TX 的配置信息。

➤ 语法

MI_S32 MI_AO_GetI2SConfig([MI_AO_If_e](#) enAoI2SIf, [MI_AUDIO_I2sConfig_t](#) *pstConfig);

➤ 形参

参数名称	描述	输入/输出
enAiI2SIf	音频I2S TX输出外设	输入
pstConfig	音频I2S TX的配置信息	输出

➤ 返回值

返回值 $\left\{ \begin{array}{ll} 0 & \text{成功。} \\ \text{非 } 0 & \text{失败，参照[错误码](#)。} \end{array} \right.$

➤ 依赖

- 头文件: mi_ao.h
- 库文件: libmi_ao.so/libmi_ao.a

※ 注意
无。

➤ 举例

```
1. MI_AUDIO_I2sConfig_t stAoI2sACfg;
2. memset(&stAoI2sACfg, 0x0, sizeof(stAoI2sACfg));
3. ExecFunc(MI_AO_GetI2SConfig(E_MI_AO_IF_I2S_A, &stAoI2sACfg), MI_SUCCESS);
```

2.21. MI_AO_AdjustSpeed

➤ 功能

调整音频输出设备的播放速度。

➤ 语法

MI_S32 MI_AO_AdjustSpeed([MI_AUDIO_DEV](#) AoDevId, MI_S32 s32Speed);

➤ 形参

参数名称	描述	输入/输出
AoDevId	音频输出设备号	输入
s32Speed	播放采样率	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败, 参照[错误码](#)。

➤ 依赖

- 头文件: mi_ao.h
- 库文件: libmi_ao.so/libmi_ao.a

※ 注意
暂不支持此接口。

➤ 举例

```
1. ExecFunc(MI_AO_AdjustSpeed(AoDevId, 8000), MI_SUCCESS);
```

2.22. MI_AO_GetTimestamp

➤ 功能

获取音频输出设备当前的播放时间戳和缓存数据量。

➤ 语法

```
MI_S32 MI_AO_GetTimestamp(MI\_AUDIO\_DEV AoDevId, MI_U32 *pu32Remaining, MI_U64 *pu64TStamp);
```

➤ 形参

参数名称	描述	输入/输出
AoDevId	音频输出设备号	输入
pu32Remaining	当前的缓存数据量	输出
pu64TStamp	当前的播放时间戳，保留未使用	输出

➤ 返回值

返回值

0	成功。
非 0	失败，参照 错误码 。

➤ 依赖

- 头文件: `mi_ao.h`
- 库文件: `libmi_ao.so/libmi_ao.a`

※ 注意

- 此接口仅能在 [MI_AO_Open](#) 成功后调用。

➤ 举例

```
1. MI_U32 u32Remaining;
2. MI_U64 u64TStamp;
3. ExecFunc(MI_AO_GetTimestamp(AoDevId, &u32Remaining, &u64TStamp), MI_SUCCESS);
```

2.23. MI_AO_GetLatency

➤ 功能

获取音频输出设备的延迟。

➤ 语法

```
MI_S32 MI_AO_GetLatency(MI\_AUDIO\_DEV AoDevId, MI_U32 *pu32Latency);
```

➤ 形参

参数名称	描述	输入/输出
AoDevId	音频输出设备号	输入
pu32Latency	延迟时间(ms)	输出

➤ 返回值

返回值

0	成功。
---	-----

返回值

非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件: `mi_ao.h`
- 库文件: `libmi_ao.so/libmi_ao.a`

※ 注意

- 此接口仅能在 [MI_AO_Open](#) 成功后调用。
- 此接口具体功能尚未实现。

➤ 举例

```
1. MI_U32 u32Latency;
2. ExecFunc(MI_AO_GetLatency(AoDevId, &u32Latency), MI_SUCCESS);
```

2.24. MI_AO_InitDev

➤ 功能

初始化音频输出设备。

➤ 语法

```
MI_S32 MI_AO_InitDev(MI_AO_InitParam_t *pstInitParam);
```

➤ 形参

参数名称	描述	输入/输出
<code>pstInitParam</code>	设备初始化参数	输入

➤ 返回值

返回值

$$\left\{ \begin{array}{ll} 0 & \text{成功。} \\ \text{非 0} & \text{失败，参照[错误码](#)。} \end{array} \right.$$

➤ 依赖

- 头文件: `mi_ao.h`
- 库文件: `libmi_ao.so/libmi_ao.a`

※ 注意

- 本接口必须和 [MI_AO_DeInitDev](#) 成对使用，不可单独重复调用，否则返回失败。
- 仅用于 STR 状态唤醒后重新初始化 AO 模块。

➤ 举例

无。

2.25. MI_AO_DeInitDev

➤ 功能

反初始化音频输出设备。

➤ 语法

```
MI_S32 MI_AO_DeInitDev(void);
```

➤ 形参

参数名称	描述	输入/输出
------	----	-------

➤ 返回值

返回值

{	0	成功。
	非 0	失败，参照 错误码 。

➤ 依赖

- 头文件: `mi_ao.h`
- 库文件: `libmi_ao.so/libmi_ao.a`

※ 注意

- 此函数必须在初始化设备后调用，否则返回失败。
- 如果本接口在 `app` 退出前没有调用，内部会自动反初始化设备。
- 本接口必须和 [MI_AO_InitDev](#) 成对使用，不可单独重复调用，否则返回失败。

➤ 举例

无。

2.26. MI_AO_GetAttr

➤ 功能

获取音频输出设备的属性。

➤ 语法

```
MI_S32 MI_AO_GetAttr(MI\_AUDIO\_DEV AoDevId, MI\_AO\_Attr\_t *pstAttr);
```

➤ 形参

参数名称	描述	输入/输出
AoDevId	音频输出设备号	输入
pstAttr	音频输出设备属性	输出

➤ 返回值

返回值

{	0	成功。
---	---	-----

非 0 失败，参照[错误码](#)。

- 依赖
 - 头文件: `mi_ao.h`
 - 库文件: `libmi_ao.so/libmi_ao.a`
- ※ 注意
 - 此函数必须在打开设备成功后调用，否则返回失败。
- 举例

无。

2.27. MI_AO_SetChannelMode

- 功能

动态设置声道输出模式。
- 语法


```
MI_S32 MI_AO_SetChannelMode(MI_AUDIO_DEV AoDevId, MI_AO_ChannelMode_e enChannelMode);
```

- 形参

参数名称	描述	输入/输出
AoDevId	音频输出设备号	输入
enChannelMode	声道输出模式	输入

- 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

- 依赖
 - 头文件: `mi_ao.h`
 - 库文件: `libmi_ao.a/libmi_ao.so`
- ※ 注意
 - 此接口仅能在 [MI_AO_AttachIf](#) 成功后调用。
 - 仅 Mochi 系列芯片支持此设定。

- 举例

简单例子如下：

```
2. ExecFunc(MI_AO_SetChannelMode(AoDevId, E_MI_AO_CHANNEL_MODE_ONLY_LEFT), MI_SUCCESS);
```

详细例子请参考 [MI_AO_Open](#)。

3. AO 数据类型

AO 模块相关数据类型定义如下：

数据类型	定义
MI_AUDIO_DEV	定义音频输出设备编号
MI_AUDIO_Format_e	定义音频输出设备的数据格式
MI_AUDIO_SoundMode_e	定义音频输出设备的声音模式
MI_AUDIO_SampleRate_e	定义音频采样率
MI_AO_ChannelMode_e	定义音频的声道输出模式
MI_AO_Attr_t	定义音频输出设备属性结构体
MI_AO_If_e	定义音频外设接口
MI_AO_GainFading_e	定义音频增益的变化速度
MI_AUDIO_I2sMode_e	定义 I2S TX 的工作模式
MI_AUDIO_I2sBitWidth_e	定义 I2S TX 的位宽
MI_AUDIO_I2sFormat_e	定义 I2S TX 的传输数据格式
MI_AUDIO_I2sMclk_e	定义 I2S TX 的 Mclk 时钟频率
MI_AUDIO_I2sConfig_t	定义 I2S TX 的相关配置信息
MI_AO_InitParam_t	定义音频设备初始化参数

3.1. MI_AUDIO_DEV

- 说明
定义音频输入/输出设备编号。
- 定义
typedef MI_S32 MI_AUDIO_DEV
- ※ 注意事项
无。
- 相关数据类型及接口
无。

3.2. MI_AUDIO_Format_e

➤ 说明

定义音频数据格式。

➤ 定义

```
typedef enum
{
    E_MI_AUDIO_FORMAT_INVALID = -1,
    E_MI_AUDIO_FORMAT_PCM_S16_LE = 0,
} MI_AUDIO_Format_e;
```

➤ 成员

成员名称	描述
E_MI_AUDIO_FORMAT_INVALID	非法的数据格式
E_MI_AUDIO_FORMAT_PCM_S16_LE	PCM Linear 16bit (Little Endian)

※ 注意事项

无。

➤ 相关数据类型及接口

[MI_AO_Attr_t](#)

3.3. MI_AUDIO_SoundMode_e

➤ 说明

定义音频声音模式。

➤ 定义

```
typedef enum
{
    E_MI_AUDIO_SOUND_MODE_MONO = 1,
    E_MI_AUDIO_SOUND_MODE_STEREO = 2,
    E_MI_AUDIO_SOUND_MODE_4CH = 4,
    E_MI_AUDIO_SOUND_MODE_6CH = 6,
    E_MI_AUDIO_SOUND_MODE_8CH = 8,
} MI_AUDIO_SoundMode_e
```

➤ 成员

成员名称	描述
E_MI_AUDIO_SOUND_MODE_MONO	单声道。
E_MI_AUDIO_SOUND_MODE_STEREO	双声道。
E_MI_AUDIO_SOUND_MODE_4CH	四声道，不支持。
E_MI_AUDIO_SOUND_MODE_6CH	六声道，不支持。

成员名称	描述
E_MI_AUDIO_SOUND_MODE_8CH	八声道，不支持。

※ 注意事项
无。

➤ 相关数据类型及接口
[MI_AO_Attr_t](#)

3.4. MI_AUDIO_SampleRate_e

➤ 说明
定义音频采样率。

➤ 定义

```
typedef enum
{
    E_MI_AUDIO_SAMPLE_RATE_8000 = 8000,
    E_MI_AUDIO_SAMPLE_RATE_11025 = 11025,
    E_MI_AUDIO_SAMPLE_RATE_12000 = 12000,
    E_MI_AUDIO_SAMPLE_RATE_16000 = 16000,
    E_MI_AUDIO_SAMPLE_RATE_22050 = 22050,
    E_MI_AUDIO_SAMPLE_RATE_24000 = 24000,
    E_MI_AUDIO_SAMPLE_RATE_32000 = 32000,
    E_MI_AUDIO_SAMPLE_RATE_44100 = 44100,
    E_MI_AUDIO_SAMPLE_RATE_48000 = 48000,
    E_MI_AUDIO_SAMPLE_RATE_96000 = 96000,
    E_MI_AUDIO_SAMPLE_RATE_192000 = 192000,
}MI_AUDIO_SampleRate_e;
```

➤ 成员

成员名称	描述
E_MI_AUDIO_SAMPLE_RATE_8000	8kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_11025	11.025kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_12000	12kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_16000	16kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_22050	22.05kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_24000	24kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_32000	32kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_44100	44.1kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_48000	48kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_96000	96kHz 采样率
E_MI_AUDIO_SAMPLE_RATE_192000	192kHz 采样率

※ 注意事项

- 除 I2S TX, 其他的音频输出外设仅支持 8/11.025/12/16/22.05/24/32/44.1/48kHz。
- I2S TX 可支持 8/16/32/48/96/192kHz, 但使用 96/192kHz 只能单独 attach I2S TX。
- Muffin 芯片 I2S TX 仅支持 8/16/32/48/96 kHz。

➤ 相关数据类型及接口

[MI_AO_Attr_t](#)

3.5. MI_AO_ChannelMode_e

➤ 说明

定义音频设备的声道输出模式。

➤ 定义

```
typedef enum
{
    E_MI_AO_CHANNEL_MODE_STEREO,
    E_MI_AO_CHANNEL_MODE_DOUBLE_MONO,
    E_MI_AO_CHANNEL_MODE_DOUBLE_LEFT,
    E_MI_AO_CHANNEL_MODE_DOUBLE_RIGHT,
    E_MI_AO_CHANNEL_MODE_EXCHANGE,
    E_MI_AO_CHANNEL_MODE_ONLY_LEFT,
    E_MI_AO_CHANNEL_MODE_ONLY_RIGHT,
}MI_AO_ChannelMode_e;
```

➤ 成员

成员名称	描述
E_MI_AO_CHANNEL_MODE_STEREO	正常立体声模式
E_MI_AO_CHANNEL_MODE_DOUBLE_MONO	立体声：左右两个声道输出为左右声道混音数据 单声道：左右两个声道输出为同样的单声道数据
E_MI_AO_CHANNEL_MODE_DOUBLE_LEFT	立体声：左右两个声道输出为左声道数据
E_MI_AO_CHANNEL_MODE_DOUBLE_RIGHT	立体声：左右两个声道输出为右声道数据
E_MI_AO_CHANNEL_MODE_EXCHANGE	立体声：左右两个声道输出为左右声道互换数据
E_MI_AO_CHANNEL_MODE_ONLY_LEFT	立体声：左声道输出为左声道数据 单声道：左声道输出为单声道数据
E_MI_AO_CHANNEL_MODE_ONLY_RIGHT	立体声：右声道输出为右声道数据 单声道：右声道输出为单声道数据

※ 注意事项

- Muffin 不支持 E_MI_AO_CHANNEL_MODE_DOUBLE_MONO(立体声) / E_MI_AO_CHANNEL_MODE_DOUBLE_LEFT(立体声) / E_MI_AO_CHANNEL_MODE_DOUBLE_RIGHT(立体声)。

➤ 相关数据类型及接口

[MI_AO_Attr_t](#)

3.6. MI_AO_Attr_t

➤ 说明

定义音频输出设备属性结构体。

➤ 定义

```
typedef struct MI_AO_Attr_s
{
    MI_AUDIO_Format_e enFormat;
    MI_AUDIO_SoundMode_e enSoundMode;
    MI_AUDIO_SampleRate_e enSampleRate;
    MI_U32 u32PeriodSize;
    MI_AO_ChannelMode enChannelMode;
    MI_AUDIO_SampleRate_e enMixerSampleRate;
} MI_AO_Attr_t;
```

➤ 成员

成员名称	描述
enFormat	音频数据格式。 静态属性。
enSoundMode	音频声音模式。 静态属性。
enSampleRate	音频采样率。 静态属性。
u32PeriodSize	起播条件的样本数。 静态属性。
enChannelMode	声道输出模式。 静态属性。
enMixerSampleRate	混音后输出采样率。 静态属性，仅在混音的使用场景时生效，如果未设定该参数，默认为 48K，其他场景下无需设定（暂不支持）

※ 注意事项

无。

- 相关数据类型及接口

[MI_AO_Open](#)

3.7. MI_AO_If_e

- 说明

定义音频外设接口。

- 定义

```
typedef enum
{
    E_MI_AO_IF_NONE = 0x0u,
    E_MI_AO_IF_DAC_AB = 0x1u,
    E_MI_AO_IF_DAC_CD = 0x02u,
    E_MI_AO_IF_I2S_A = 0x4u,
    E_MI_AO_IF_I2S_B = 0x8u,
    E_MI_AO_IF_ECHO_A = 0x10u,
    E_MI_AO_IF_HDMI_A = 0x20u,
    E_MI_AO_IF_MAX,
} MI_AO_If_e;
```

- 成员

成员名称	描述
E_MI_AO_IF_NONE	None
E_MI_AO_IF_DAC_AB	DAC0/1
E_MI_AO_IF_DAC_CD	DAC2/3
E_MI_AO_IF_I2S_A	I2S TX A
E_MI_AO_IF_I2S_B	I2S RX B
E_MI_AO_IF_ECHO_A	SRC 数据（回声）
E_MI_AO_IF_HDMI_A	HDMI TX

- ※ 注意事项

无。

- 相关数据类型及接口

[MI_AO_AttachIf](#)

[MI_AO_DetachIf](#)

[MI_AO_SetIfVolume](#)

[MI_AO_GetIfVolume](#)

[MI_AO_SetIfMute](#)

[MI_AO_GetIfMute](#)

[MI_AO_SetI2SConfig](#)

[MI_AO_GetI2SConfig](#)

3.8. MI_AO_GainFading_e

➤ 说明

定义音频设备增益的变化速度。

➤ 定义

```
typedef enum{
    E_MI_AO_GAIN_FADING_OFF = 0,
    E_MI_AO_GAIN_FADING_1_SAMPLE,
    E_MI_AO_GAIN_FADING_2_SAMPLE,
    E_MI_AO_GAIN_FADING_4_SAMPLE,
    E_MI_AO_GAIN_FADING_8_SAMPLE,
    E_MI_AO_GAIN_FADING_16_SAMPLE,
    E_MI_AO_GAIN_FADING_32_SAMPLE,
    E_MI_AO_GAIN_FADING_64_SAMPLE,
}MI_AO_GainFading_e;
```

➤ 成员

成员名称	描述
E_MI_AO_GAIN_FADING_OFF	关闭Fading功能，设置增益立刻生效
E_MI_AO_GAIN_FADING_1_SAMPLE	开启Fading功能，1个采样点变化0.5dB
E_MI_AO_GAIN_FADING_2_SAMPLE	开启Fading功能，2个采样点变化0.5dB
E_MI_AO_GAIN_FADING_4_SAMPLE	开启Fading功能，4个采样点变化0.5dB
E_MI_AO_GAIN_FADING_8_SAMPLE	开启Fading功能，8个采样点变化0.5dB
E_MI_AO_GAIN_FADING_16_SAMPLE	开启Fading功能，16个采样点变化0.5dB
E_MI_AO_GAIN_FADING_32_SAMPLE	开启Fading功能，32个采样点变化0.5dB
E_MI_AO_GAIN_FADING_64_SAMPLE	开启Fading功能，64个采样点变化0.5dB

※ 注意事项

无。

➤ 相关数据类型及接口

[MI_AO_SetVolume](#)

[MI_AO_SetMute](#)

3.9. MI_AUDIO_I2sMode_e

➤ 说明

定义音频 I2S RX/TX 的工作模式。

➤ 定义

```
typedef enum
{
    E_MI_AUDIO_I2S_MODE_I2S_MASTER,
    E_MI_AUDIO_I2S_MODE_I2S_SLAVE,
    E_MI_AUDIO_I2S_MODE_TDM_MASTER,
    E_MI_AUDIO_I2S_MODE_TDM_SLAVE,
} MI_AUDIO_I2sMode_e;
```

➤ 成员

成员名称	描述
E_MI_AUDIO_I2S_MODE_I2S_MASTER	I2S 主模式
E_MI_AUDIO_I2S_MODE_I2S_SLAVE	I2S 从模式
E_MI_AUDIO_I2S_MODE_TDM_MASTER	TDM 主模式
E_MI_AUDIO_I2S_MODE_TDM_SLAVE	TDM 从模式

※ 注意事项

- 主模式与从模式是否支持会依据不同的芯片而有区别。

➤ 相关数据类型及接口

[MI_AUDIO_I2sConfig_t](#)

3.10. MI_AUDIO_I2sBitWidth_e

➤ 说明

定义音频 I2S RX/TX 的位宽。

➤ 定义

```
typedef enum
{
    E_MI_AUDIO_BIT_WIDTH_16,
    E_MI_AUDIO_BIT_WIDTH_32,
} MI_AUDIO_I2sBitWidth_e;
```

➤ 成员

成员名称	描述
E_MI_AUDIO_BIT_WIDTH_16	I2S 位宽为 16bit。
E_MI_AUDIO_BIT_WIDTH_32	I2S 位宽为 32bit。

※ 注意事项

无。

➤ 相关数据类型及接口

[MI_AUDIO_I2sConfig_t](#)

3.11. MI_AUDIO_I2sFormat_e

➤ 说明

定义音频 I2S RX/TX 的数据传输格式。

➤ 定义

```
typedef enum
{
    E_MI_AUDIO_I2S_FMT_I2S_MSB,
    E_MI_AUDIO_I2S_FMT_LEFT_JUSTIFY_MSB,
} MI_AUDIO_I2sFormat_e;
```

➤ 成员

成员名称	描述
E_MI_AUDIO_I2S_FMT_I2S_MSB	I2S 标准格式，最高位优先
E_MI_AUDIO_I2S_FMT_LEFT_JUSTIFY_MSB	I2S 左对齐格式，最高位优先

※ 注意事项

无

➤ 相关数据类型及接口

[MI_AUDIO_I2sConfig_t](#)

3.12. MI_AUDIO_I2sMclk_e

➤ 说明

定义音频 I2S RX/TX 的 mclk 的时钟频率。

➤ 定义

```
typedef enum
{
    E_MI_AUDIO_I2S_MCLK_0,
    E_MI_AUDIO_I2S_MCLK_12_288M,
    E_MI_AUDIO_I2S_MCLK_16_384M,
    E_MI_AUDIO_I2S_MCLK_18_432M,
    E_MI_AUDIO_I2S_MCLK_24_576M,
    E_MI_AUDIO_I2S_MCLK_24M,
    E_MI_AUDIO_I2S_MCLK_48M,
} MI_AUDIO_I2sMclk_e;
```

➤ 成员

成员名称	描述
------	----

成员名称	描述
E_MI_AUDIO_I2S_MCLK_12_288M	设置 MCLK 为 12.88M
E_MI_AUDIO_I2S_MCLK_16_384M	设置 MCLK 为 16.384M
E_MI_AUDIO_I2S_MCLK_18_432M	设置 MCLK 为 18.432M
E_MI_AUDIO_I2S_MCLK_24_576M	设置 MCLK 为 24.576M
E_MI_AUDIO_I2S_MCLK_24M	设置 MCLK 为 24M
E_MI_AUDIO_I2S_MCLK_48M	设置 MCLK 为 48M

※ 注意事项
无

➤ 相关数据类型及接口
[MI_AUDIO_I2sConfig_t](#)

3.13. MI_AUDIO_I2sConfig_t

➤ 说明
定义 I2S RX/TX 的配置信息。

➤ 定义

```
typedef struct MI_AUDIO_I2sConfig_s
{
    MI_AUDIO_I2sMode_e enMode;
    MI_AUDIO_I2sBitWidth_e enBitWidth;
    MI_AUDIO_I2sFormat_e enFormat;
    MI_AUDIO_SampleRate_e enSampleRate;
    MI_AUDIO_I2sMclk_e enMclk;
    MI_BOOL bSyncClock;
    MI_U32 u32TdmSlots;
} MI_AUDIO_I2sConfig_t;
```

➤ 成员

成员名称	描述
enMode	I2S 的工作模式。
enBitWidth	I2S 的数据位宽。
enFormat	I2S 的传输数据格式。
enSampleRate	I2S 的采样率。
enMclk	I2S 的 mclk 时钟频率。
bSyncClock	I2S RX 和 I2S TX 是否共用时钟。
u32TdmSlots	I2S TDM slot 的数目（仅 TDM 模式下有效）

※ 注意事项

无。

- 相关数据类型及接口
 - [MI_AO_SetI2SConfig](#)
 - [MI_AO_GetI2SConfig](#)

3.14. MI_AO_InitParam_t

- 说明

音频输出设备初始化参数。
- 定义

```
typedef struct MI_AO_InitParam_s
{
    MI_AUDIO_DEV AoDevId;
    MI_U8 *u8Data;
} MI_AO_InitParam_t;
```

- 成员

成员名称	描述
AoDevId	音频输出设备号
u8Data	参数指针（保留）

- ※ 注意事项

无。

- 相关数据类型及接口
 - [MI_AO_InitDev](#)

4. 错误码

AO API 错误码如下表所示:

错误码	宏定义	描述
0xA0052001	MI_AO_ERR_INVALID_DEVID	音频输出设备号无效
0xA0052003	MI_AO_ERR_ILLEGAL_PARAM	音频输出参数设置无效
0xA0052006	MI_AO_ERR_NULL_PTR	输入参数空指针错误
0xA0052007	MI_AO_ERR_NOT_CONFIG	音频输出设备属性未设置
0xA0052008	MI_AO_ERR_NOT_SUPPORT	操作不支持
0xA0052009	MI_AO_ERR_NOT_PERM	操作不允许
0xA005200C	MI_AO_ERR_NOMEM	分配内存失败
0xA005200D	MI_AO_ERR_NOBUF	音频输出缓存不足
0xA005200E	MI_AO_ERR_BUF_EMPTY	音频输出缓存为空
0xA005200F	MI_AO_ERR_BUF_FULL	音频输出缓存为满
0xA0052010	MI_AO_ERR_SYS_NOTREADY	音频输出系统未初始化
0xA0052012	MI_AO_ERR_BUSY	音频输出系统忙碌
0xA0052017	MI_AO_ERR_NOT_ENABLED	音频输出设备或信道没有使能
0xA005201D	MI_AO_ERR_NOVASPACE	音频设备映射 buffer 失败

5. PROCFS 介绍

5.1. cat

➤ 调试信息

cat proc/mi_modules/mi_ao/mi_ao0

```
===== MI AO Dev0 Info =====

----- Start AO Dev0 Attr -----
DevStatus   Format   SoundMode  SampleRate  PeriodSize  ChannelMode  MixerSampleRate
opened      S16_LE   Stereo     8000        1024        Stereo       48000

DmaBufSize  DmaBusySize  DmaFreeSize  TmpBufSize  bStart  bPause  FrmCnt
65536       62320       3216        65536      1       0       73

DpgaGain    DpgaMute
(0,0)       (0,0)

MI If: DAC_AB
Mhal Left If: DAC_A_0
Mhal Right If: DAC_B_0

If Info:
DAC_AB      : Volume( 0, 0) Mute(0,0) bIsMixing:0
DAC_CD      : Volume( 0, 0) Mute(0,0) bIsMixing:0
I2S_A       : Volume( 0, 0) Mute(0,0) bIsMixing:0
I2S_B       : Volume( 0, 0) Mute(0,0) bIsMixing:0
ECHO_A      : Volume( 0, 0) Mute(0,0) bIsMixing:0
HDMI_A      : Volume( 0, 0) Mute(0,0) bIsMixing:0

----- End AO Dev0 Attr -----
```

➤ 调试信息分析

记录当前 AO 的使用状况以及 device 属性等，可以动态地获取到这些信息，方便调试和测试。

➤ 参数说明

参数		描述
AO Device Attr (AO 设备属性)	DevStatus	AO 设备状态 uninit:未初始化 opened:open 成功
	Format	音频格式（位宽及大小端等） 目前仅支持 S16_LE（16bit，小端模式）
	SoundMode	声音模式 mono: 单声道 stereo: 立体声
	SampleRate	采样率 8k/11.025k/12k/16k/22.05k/24k/32k/44.1k/48k
	PeriodSize	AO 起播水线（采样点数）
	ChannelMode	AO 通道模式

参数		描述
		Stereo: 立体声 DoubleMono: 目前仅支持, 播 MONO-左右声道输出相同 DoubleLeft: 输出两路左声道 DoubleRight: 输出两路右声道 Exchange: 左右声道交换 OnlyLeft: 目前仅支持 MONO-左声道输出 Only Right: 目前仅支持 MONO-右声道输出
	MixerSampleRate	混音后降频输出采样率 8k/16k/32k/48k (暂时无效)
	DmaBufSize	DMA buffer 大小
	DmaBusySize	DMA buffer 中已使用的大小
	DmaFreeSize	DMA buffer 中未使用的大小
	TmpBufSize	Tmp 临时 buffer 大小
	bStart	AO 是否开启 DMA
	bPause	AO 是否暂停 DMA
	FrmCnt	写数据帧计数
	DpgaGain	数字增益: (左声道增益, 右声道增益)
	DpgaMute	DPGA 静音: (左声道静音, 右声道静音)
	MI If	MI 绑定的所有 interface 信息, 即应用层绑定的所有 interface 信息
	Mhal Left If	MHAL RDMA_L 绑定的所有 interface 信息
	Mhal Right If	MHAL RDMA_R 绑定的所有 interface 信息
	If Info	Interface 音量、静音等信息 Volume (左声道增益, 右声道增益) Mute (左声道静音, 右声道静音)
AO I2S Status (AO I2S 信息)	I2sMode	I2S Tx 的工作模式(仅 interface 为 I2S Tx 时有效) i2s-master i2s-slave tdm-master tdm-slave
	I2sMclk	I2S Tx 的 Mclk 的频率(仅 interface 为 I2S Tx 时有效) disable: 不使用 Mclk 其他值为当前的 Mclk 频率
	I2sFmt	I2S Tx 的数据格式(仅 interface 为 I2S Tx 时有效) I2S-MSB: I2S 格式 LEFT-MSB: I2S 左对齐格式
	bI2sSync	I2S RX 和 TX 是否共用 clock(仅 interface 为 I2S Tx 时有效) 1: 4 wire mode, RX 和 TX 共用 clock 0: 6 wire mode, RX 和 TX 都有独立的 clock
	TdmSlots	I2S Tx 的 TDM slot 数目(仅 interface 为 I2S Tx 为 TDM 模式时有效)
	I2sBitWidth	I2S TX 的位宽(仅 interface 为 I2S Tx, 且需要支持 TDM 模

参数	描述
	式的芯片有效)

5.2. echo

功能	动态启用/关闭 AO 设备 DPGA 静音模式
命令	echo set_dpga_mute [LeftMute] [RightMute] [Fading] > proc/mi_modules/mi_ao/mi_ao[ID]
参数说明	[ON/on/1, OFF/off/0] 开启/关闭静音 [Fading]设置渐入渐出速度 [ID] 设备号
举例	echo set_dpga_mute 1 0 0 > proc/mi_modules/mi_ao/mi_ao[ID]

功能	动态启用/关闭 AO 设备 Interface 静音模式
命令	echo set_inf_mute [If] [LeftMute] [RightMute] > proc/mi_modules/mi_ao/mi_ao[ID]
参数说明	[If]需要设定的 Interface [ON/on/1, OFF/off/0] 开启/关闭静音 [ID] 设备号
举例	echo set_inf_mute 1 1 0 > proc/mi_modules/mi_ao/mi_ao[ID]

功能	动态修改 AO DPGA 音量大小
命令	echo set_dpga_volume [LeftVolume] [RightVolume] [Fading] > proc/mi_modules/mi_ao/mi_ao[ID]
参数说明	[LeftVolume] [RightVolume] 左声道音量，右声道音量 [Fading]设置音量渐入渐出速度
举例	echo set_dpga_volume -10 -10 0 > proc/mi_modules/mi_ao/mi_ao0

功能	动态修改 AO Interface 音量大小
命令	echo set_inf_volume [If] [LeftVolume] [RightVolume] > proc/mi_modules/mi_ao/mi_ao[ID]
参数说明	[If]需要设定的 Interface [LeftVolume] [RightVolume] 左声道音量，右声道音量
举例	暂不支持

功能	动态开启/关闭 AO 设备 dump 数据功能
命令	echo dump_data [Path] [ON/on/1, OFF/off/0] > proc/mi_modules/mi_ao/mi_ao[ID]
参数说明	[Path] dump 数据的保存路径 [ON/on/1, OFF/off/0] 开启/关闭 dump 数据功能
举例	echo dump /mnt 1 > proc/mi_modules/mi_ao/mi_ao0

功能	动态开启/关闭 AO 设备的 Singen 功能
命令	echo singen [SinGen Index] [Enable] > proc/mi_modules/mi_ao/mi_ao[ID]
参数说明	[SinGen Index] 需要控制的 Singen ID
	[Enable] 开启/关闭 Singen 功能
举例	echo singen 0 1 > proc/mi_modules/mi_ao/mi_ao0

功能	动态设置 AO 设备的 channel mode
命令	echo set_chn_mode [channelMode] > proc/mi_modules/mi_ao/mi_ao[ID]
参数说明	[channelMode] 需要设定的 channel mode
举例	echo set_chn_mode 0 > proc/mi_modules/mi_ao/mi_ao0